# SMART CLASSROOM AUTOMATION SYSTEM

*A Mini Project Report submitted to the Microprocessor lab*

*Submitted by*

| Registration Number | Sec-RollNo | Name |
|---|---|---|
| | | |
| | | |
| | | |
| | | |

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING**

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
*(A constituent unit of MAHE, Manipal)*

MANIPAL-576104, KARNATAKA, INDIA

**NOVEMBER 2026**

# ABSTRACT

Classroom lighting and ventilation are conventionally controlled from a common switchboard, so these loads frequently remain energised over unoccupied areas and after the room has been vacated. Commercial occupancy-based controllers address this problem but are expensive for institutional classrooms. The objective of this project was a low-cost, fault-tolerant controller that energises each section of a classroom only when it is in use.

The classroom is divided into two zones, each monitored by an HC-SR04 ultrasonic sensor and served by an independent light, which is enabled only under occupancy and low ambient light reported by an LDR module. Occupancy is filtered with delays of 5 s for activation and 10 s for deactivation. An RC522 RFID reader at the entrance maintains a check-in count, and a DC fan driven through a logic-level MOSFET operates only while this count is non-zero. Potentiometers set light intensity and fan speed, a push button provides manual override through an external interrupt, and the on-chip RTC timestamps events. The firmware was implemented in bare-metal C on an STM32F401RE.

All target specifications were met on hardware. Lighting responded immediately to check-in, each zone light followed its occupancy and ambient-light conditions, and disconnection of any sensor or the display did not halt operation. The firmware uses eight on-chip peripherals and two interrupt sources in approximately 17 KB of program memory, and its decision logic passed 460 automated test cases.

The results show that zone-level classroom automation is achievable with inexpensive modules on a single microcontroller when every hardware access is time-bounded and every fault is reported. The principal limitation is that RFID counts credentials rather than persons. Tools used: STM32 Nucleo-F401RE, GNU Arm Embedded Toolchain, CMake, STM32CubeProgrammer, Visual Studio Code, serial terminal and Wokwi.

# Contents


| | | | Page No |
|---|---|---|---|
| Abstract | | | ii |
| **Chapter 1** | | **INTRODUCTION** | **1** |
| | **1.1** | Introduction | 1 |
| | **1.2** | Review of Literature | 1 |
| | **1.3** | Objectives | 2 |
| | **1.4** | Target Specifications | 2 |
| | **1.5** | Organization of the Report | 3 |
| | | | |
| **Chapter 2** | | **METHODOLOGY** | **4** |
| | **2.1** | Introduction | 4 |
| | **2.2** | Development Approach | 4 |
| | **2.3** | Block Diagram and Circuit Layout | 5 |
| | **2.4** | Component Specifications and Selection | 8 |
| | **2.5** | Control Algorithm | 9 |
| | **2.6** | Firmware Design | 13 |
| | **2.7** | Tools Used | 16 |
| | **2.8** | Preliminary Results | 17 |
| | **2.9** | Conclusions | 17 |
| | | | |
| **Chapter 3** | | **RESULT ANALYSIS** | **18** |
| | **3.1** | Introduction | 18 |
| | **3.2** | Photographs of the Working Project | 18 |
| | **3.3** | Result Analysis | 19 |
| | **3.4** | Significance of the Results | 21 |
| | **3.5** | Deviations from Expected Results and Justification | 21 |
| | **3.6** | Conclusions | 24 |
| | | | |
| **Chapter 4** | | **CONCLUSION AND FUTURE SCOPE OF WORK** | **25** |
| | **4.1** | Work Conclusion | 25 |
| | **4.2** | Future Scope of Work | 25 |
| | | | |
| **REFERENCES** | | | 26 |
| **ANNEXURES** | | | 27 |
| **Plagiarism Report** | | | 45 |

# CHAPTER 1

# INTRODUCTION

## 1.1 Introduction

This chapter describes the problem of energy wastage in classroom lighting and ventilation, surveys related work on occupancy-based control and RFID-based room management, and states the objectives and target specifications of the project. The organisation of the report is outlined at the end of the chapter.

In a conventional classroom, all lights and fans are controlled from a common switchboard located near the entrance. Once energised, these loads typically remain on for the duration of occupancy irrespective of the number of seats in use or the availability of daylight, and are frequently left on during intervals and after the room has been vacated. Although the energy consumed per room per hour is modest, the cumulative wastage across an institution over a working day is significant.

Occupancy-based load control addresses this problem and is available commercially; however, such systems are designed and priced for office and commercial buildings. A controller intended for institutional classrooms with limited budgets must instead be built from inexpensive and readily replaceable modules, must be simple to install and maintain, and must continue to operate in a safe state when an individual component fails.

## 1.2 Review of Literature

Floyd, Parker and Sherwin reported field measurements of occupancy sensors installed in three buildings, two of which were elementary schools [1]. At one school, the sensors reduced lighting energy by an average of 10.8 % on school days, with most of the reduction occurring in the evening; at the other, lighting energy increased after installation and commissioning. The study indicates that the savings obtained depend strongly on sensor configuration and on the usage pattern of the space.

Guo, Tiller, Henze and Waters reviewed the performance of occupancy-based lighting control systems [2]. The review examines the available sensing technologies and the influence of time-delay settings and sensor coverage on both energy savings and occupant satisfaction. It notes that short time delays cause lights to be switched off while occupants are stationary, which reduces acceptance of the system. This observation motivated the use of asymmetric delays in the present work.

Ani et al. proposed an IoT-based smart classroom in which the room is divided into two segments and lights and fans in each segment are switched according to occupancy detected by a camera [3]. The segmentation of the room is analogous to the zoning adopted in this project;

however, camera-based detection requires substantial processing capability, increases cost, and raises privacy concerns in an educational setting.

Ibrahim et al. developed an IoT-based classroom control system on an Arduino platform, in which infrared sensors at the entrance detect persons entering and leaving and the lights and fans are switched accordingly [4]. The approach treats the room as a single load and relies on accurate counting of every entry and exit.

Mijic and Bjelica described an RFID-based system for higher-education classrooms that automates attendance recording, counts teaching sessions and logs the use of classroom resources, with the data integrated into an institutional quality-assurance system [5]. The work establishes RFID as a practical means of identifying the occupants of a room, although its purpose is record keeping rather than control of electrical loads.

The surveyed literature leads to three conclusions. First, occupancy control yields energy savings only when time delays are selected with occupant behaviour in mind. Second, zoning provides finer control than switching the room as a whole. Third, entry-based identification such as RFID is informative but cannot substitute for direct sensing of occupant location. The present project therefore combines zone-level ultrasonic sensing for lighting with RFID-based check-in for ventilation, implemented on a low-cost microcontroller without cameras, cloud services or network connectivity.

## 1.3 Objectives

### *1.3.1 Main objective*

To design and implement a microcontroller-based controller that energises the light of each zone only when that zone is occupied and ambient light is insufficient, and that operates the fan only while at least one person is checked in at the entrance.

### *1.3.2 Secondary objectives*

- To energise the lighting immediately upon check-in at the entrance, without waiting for the zone occupancy filter to settle.
- To provide user adjustment of light intensity and fan speed, and a manual override for both loads.
- To display the system state locally and to log all events with a timestamp.
- To ensure continued operation when any sensor or the display is disconnected.
- To implement all firmware at register level using the GPIO, timer, ADC, I2C, SPI, UART, RTC and interrupt peripherals of the microcontroller.

## 1.4 Target Specifications

The specifications adopted for the final system are listed in Table 1.1.

**Table 1.1:** Target specifications

| Parameter | Target |
|---|---|
| Number of zones | 2, each with an independent sensor and light |
| Occupancy detection range | Object within 100 cm of the zone sensor |
| Activation delay | 5 s of continuous detection |
| Deactivation delay | 10 s of continuous absence |
| Lighting in automatic mode | Enabled only when the zone is occupied and ambient light is low |
| Light intensity control | Continuous, approximately 5–100 %, by potentiometer |
| Entrance check-in | RFID check-in and check-out, up to 8 credentials |
| Fan control | Enabled only while at least one credential is checked in; speed by potentiometer |
| Manual override | Single push button: automatic, all on, all off |
| Display refresh | At least once per second |
| Fault tolerance | No loss of operation on disconnection of any sensor or the display |
| Firmware | Bare-metal C without HAL or Arduino libraries |

**Importance of the end result.** A controller meeting these specifications de-energises loads over unoccupied areas without inconveniencing occupants, and can be constructed from components costing a few hundred rupees in addition to the microcontroller. Since every hardware access is time-bounded and every fault condition is reported, the system can be maintained with a multimeter and a serial terminal.

## 1.5 Organization of the Report

Chapter 2 presents the methodology, comprising the selection of the hardware platform, the block diagram and circuit layout, the component specifications and their justification, the control algorithm, the design of each firmware driver and the tools employed. Chapter 3 presents the results, including photographs of the working prototype, functional test results, timing and memory figures, and an analysis of the deviations observed during development together with their resolution. Chapter 4 summarises the work and identifies the scope for future development. The references and the complete firmware source are provided at the end of the report.

# CHAPTER 2

# METHODOLOGY

## 2.1 Introduction

This chapter describes the development process and the design of the system. The development approach and the evolution of the hardware platform are presented first, followed by the block diagram, pin allocation and motor driver circuit. The component specifications and the rationale for their selection are then given, after which the control algorithm and the design of each firmware module are described. The chapter concludes with the tools used and the results obtained at the mid-semester stage.

## 2.2 Development Approach

The hardware platform was revised twice before the final design was reached, as summarised in Fig. 2.1. The control logic was initially developed for an STM32F103 (Bluepill) board in the Wokwi simulator, which allowed the zone logic, the 100 cm detection threshold and the lighting modes to be verified before any components were procured.

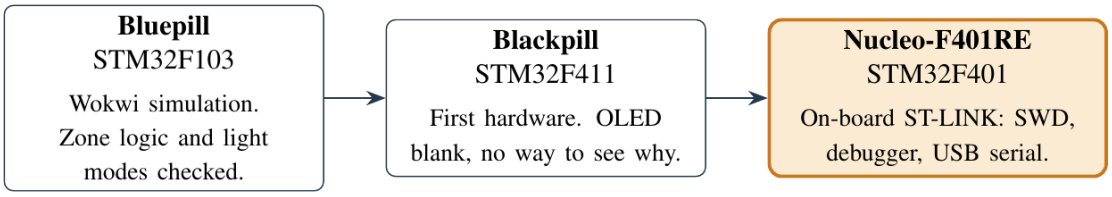


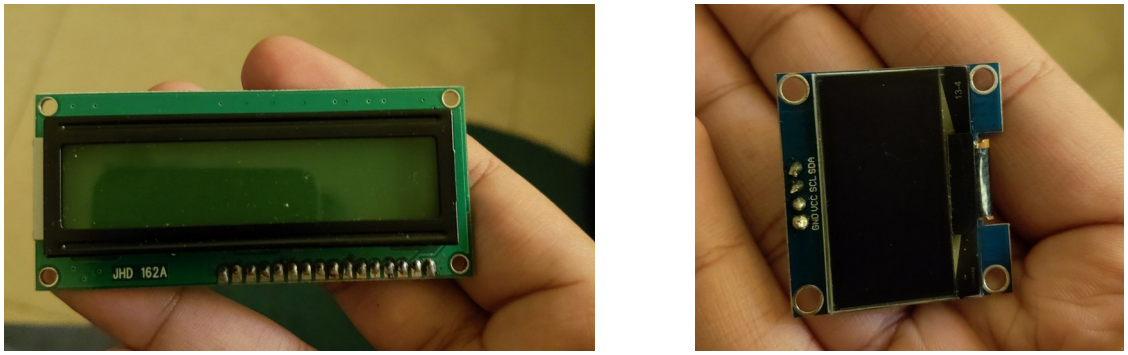

**Fig. 2.1:** Evolution of the development platform

The first hardware implementation used a Blackpill (STM32F411) board. On this platform the OLED display failed to initialise, and the cause could not be determined because the board provides neither an on-board debugger nor a USB-to-serial interface. Development was therefore moved to the STM32 Nucleo-F401RE [6], which integrates an ST-LINK debugger providing SWD programming, breakpoint debugging and a USB virtual COM port connected to USART2. Two diagnostic aids were adopted from this point onward: a serial status line reporting the complete system state on every pass of the main loop, and a heartbeat obtained by toggling the on-board LED LD2 once per pass. As the STM32F401 and STM32F411 share the STM32F4 register map [7], the drivers required only pin reassignment when ported.

During initial bring-up, a malfunction could originate either in the new wiring or in the new register-level firmware. To isolate these causes, each feature was first implemented as an STM32duino (Arduino) sketch using established libraries. Once a feature operated correctly on the actual wiring, any subsequent fault in the bare-metal implementation could be attributed to the firmware. Each feature was validated in this manner and then reimplemented at register level; the final firmware contains no Arduino or HAL code.

The initial design comprised three zones. The sensor of the third zone consistently returned a no-echo condition, including after replacement, while identical code operated correctly for the remaining two zones. Since the driver distinguishes a sensor that does not respond from one that detects no object within range, the fault was attributed to wiring rather than to an unoccupied zone. The design was consequently reduced to two zones, which simplifies wiring and reduces cost per room. Zone parameters are stored in arrays in the firmware, so additional zones can be supported by extending the pin tables.

A character LCD was originally planned for local display; the final implementation uses a 1.3-inch 128×64 OLED module with an SH1106 controller (Fig. 2.2). A dedicated driver was written for this controller, since it differs from the more widely supported SSD1306.




*(a) JHD162A character LCD* *(b) 1.3-inch SH1106 OLED (used)*

**Fig. 2.2:** Display modules considered; the OLED module was used in the final implementation

## 2.3 Block Diagram and Circuit Layout

### *2.3.1 System block diagram*

The block diagram of the system is shown in Fig. 2.3. The Nucleo-F401RE forms the central controller, and each connection corresponds to an on-chip peripheral. The inputs are the two ultrasonic sensors, the LDR module, the DHT11 sensor, the RFID reader, the two potentiometers and the push button. The outputs are the two zone lights, the fan motor, the OLED display and a serial link to a host computer; the serial link is the only bidirectional connection.

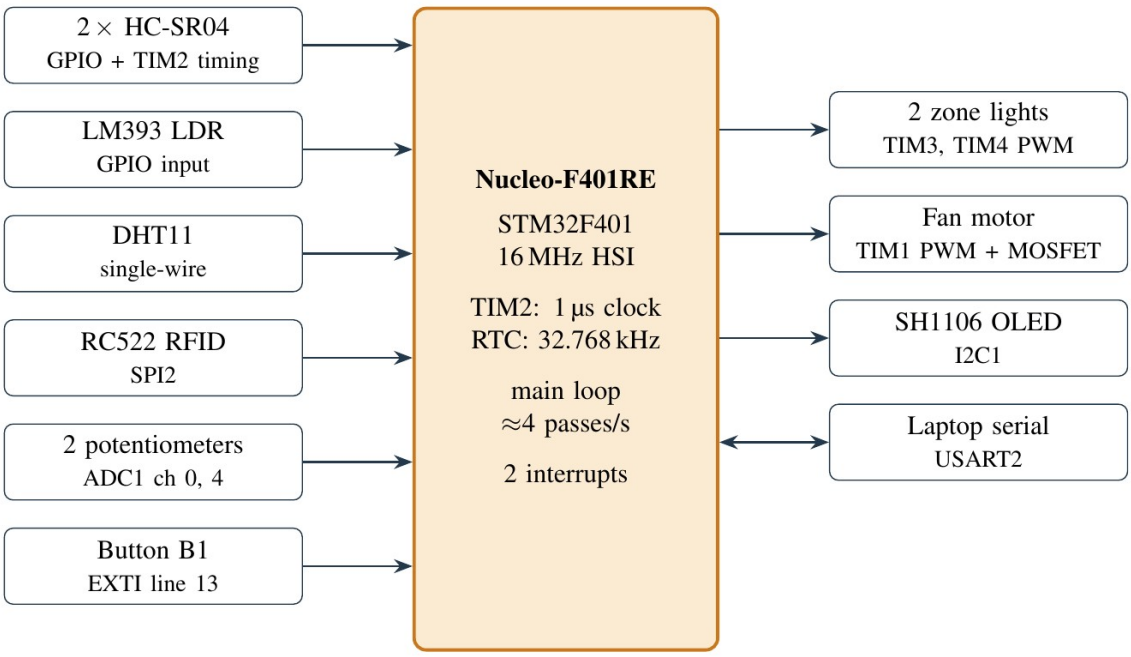



**Fig. 2.3:** System block diagram

### *2.3.2 Pin allocation*

The pin allocation is given in Table 2.1. Header designations such as D9 and A0 refer to the Arduino-compatible connectors of the Nucleo board, while PB12–PB15 and PC5 are available only on the ST morpho connectors. All modules are supplied from the 3.3 V rail of the board except the ultrasonic sensors, which require 5 V, and the motor, which is supplied from a separate 9 V battery. All grounds, including the negative terminal of the battery, are connected together.

**Table 2.1:** Pin and peripheral allocation

| Signal | Header | MCU pin | Peripheral / function |
|---|---|---|---|
| Zone A trigger / echo | D2 / D3 | PA10 / PB3 | GPIO out / GPIO in |
| Zone B trigger / echo | D4 / D5 | PB5 / PB4 | GPIO out / GPIO in |
| Zone A light | D9 | PC7 | TIM3 CH2 (AF2), PWM |
| Zone B light | D10 | PB6 | TIM4 CH1 (AF2), PWM |
| DHT11 data | D8 | PA9 | GPIO, open-drain |
| OLED SCL / SDA | D15 / D14 | PB8 / PB9 | I2C1 (AF4) |
| LDR module output | A3 | PB0 | GPIO in, pull-up |
| RC522 SCK / MISO / MOSI | morpho | PB13 / PB14 / PB15 | SPI2 (AF5) |
| RC522 chip select / reset | morpho | PB12 / PC5 | GPIO out |
| Light potentiometer | A0 | PA0 | ADC1 channel 0 |
| Fan potentiometer | A2 | PA4 | ADC1 channel 4 |
| Fan MOSFET gate | D7 | PA8 | TIM1 CH1 (AF1), PWM |
| Override button B1 | on board | PC13 | EXTI line 13 |

| Signal | Header | MCU pin | Peripheral / function |
|---|---|---|---|
| Heartbeat LED LD2 | D13 | PA5 | GPIO out |
| USB serial TX / RX | on board | PA2 / PA3 | USART2 (AF7) |
| RTC crystal | on board | PC14 / PC15 | LSE oscillator |

The echo output of the HC-SR04 has a 5 V logic level. PB3 and PB4 are 5 V-tolerant inputs [8], and level shifting is therefore not required. The RC522 module, in contrast, must be supplied from 3.3 V only. Each potentiometer is connected as a voltage divider between 3.3 V and ground, with the wiper connected to the analog input; a 5 V level must not be applied to any pin configured in analog mode.

### *2.3.3 Motor driver circuit*

A GPIO pin of the microcontroller can source approximately 20 mA at 3.3 V, whereas the type-130 DC motor used to represent the fan draws several hundred milliamperes at a rated voltage of 3–6 V. The motor is therefore switched by a logic-level N-channel MOSFET placed on the low side of the load, with the gate driven by the PWM output (Fig. 2.4).

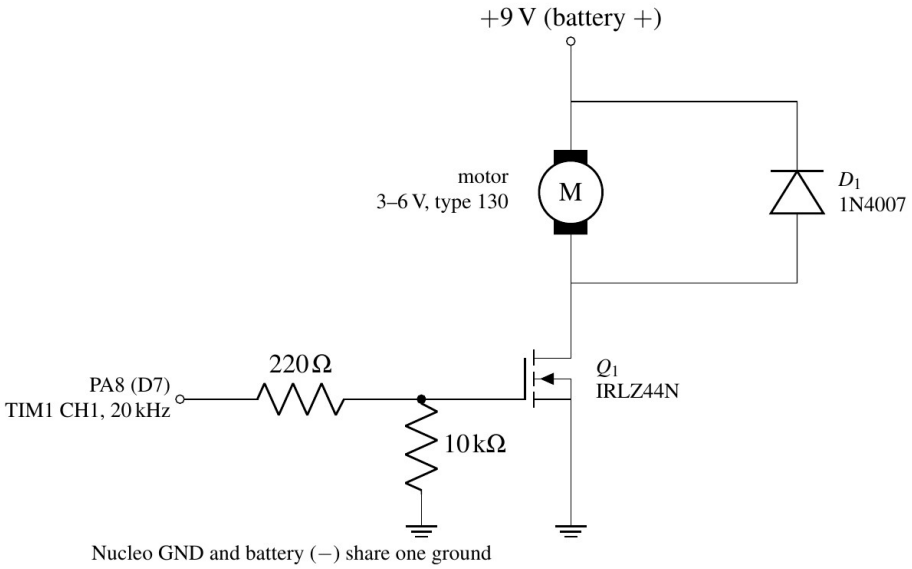



**Fig. 2.4:** Fan motor driver using a low-side logic-level MOSFET with flyback diode

- **Q1 (IRLZ44N).** The on-resistance of this device is specified at a gate-source voltage of 4–5 V, and it is fully enhanced by the 3.3 V output of the microcontroller [9]. Standard-threshold devices such as the IRFZ44N and IRF540N require approximately 10 V for full enhancement and are unsuitable for direct drive.
- **D1 (1N4007).** The motor winding is inductive. When Q1 turns off, the winding current cannot change instantaneously, and in the absence of a conduction path a voltage transient sufficient to damage Q1 or disturb the controller through the common ground would be produced. D1, connected with its cathode to the +9 V supply, provides a freewheeling path for this current.

- **Gate resistor (220 Ω).** Limits the charging current of the gate capacitance of Q1 at each PWM transition.
- **Gate pull-down resistor (10 kΩ).** Holds the gate at 0 V while the output pin is unconfigured during reset and programming, preventing unintended motor operation at power-up.
- **Common ground.** The negative terminal of the battery is connected to the ground of the Nucleo board to provide a common reference for the gate drive. The positive terminal of the battery is isolated from the controller and from the sensor supply rails.

## 2.4 Component Specifications and Selection

The components used and their principal specifications are listed in Table 2.2. Unit prices correspond to the parts order placed on 13 September 2026; the OLED module, LDR module, RFID kit and potentiometers were procured separately.

**Table 2.2:** Component specifications

| Component | Qty | Key specification | Unit price (Rs.) |
|---|---|---|---|
| STM32 Nucleo-F401RE | 1 | STM32F401RE, Cortex-M4, up to 84 MHz, 512 KB flash, 96 KB RAM; on-board ST-LINK/V2-1 | 2579 |
| HC-SR04 ultrasonic sensor | 2 | 5 V, 40 kHz, range approximately 2–400 cm, trigger/echo interface | 69 |
| DHT11 | 1 | 3.3–5 V, single-wire interface, resolution 1 °C and 1 % RH | 51 |
| LED module, 5 mm | 2 | Represents the lighting of each zone | 51 |
| DC motor, type 130 | 1 | 3–6 V, represents the fan | 45 |
| IRLZ44N MOSFET | 1 | Logic-level N-channel, 55 V, TO-220 | 39 |
| 1N4007 diode | 1 | 1 A, 1000 V general-purpose rectifier | 12 |
| Resistors | 2 | 220 Ω (gate), 10 kΩ (pull-down) | < 2 |
| LM393 LDR module | 1 | Light-dependent resistor with comparator, digital output | – |
| SH1106 OLED module | 1 | 1.3-inch, 128×64, I2C address 0x3C/0x3D | – |
| RC522 RFID kit | 1 | 13.56 MHz ISO/IEC 14443A reader with S50 card and key tag | – |
| Potentiometer, 10 kΩ | 2 | Light intensity and fan speed adjustment | – |
| 9 V battery | 1 | Motor supply, isolated from the logic supply | – |

### *2.4.1 Justification for component selection*

- **Nucleo-F401RE.** The integrated ST-LINK provides programming, debugging and serial communication without additional hardware, which was the primary reason for its selection following the difficulties encountered with the Blackpill. The STM32F401 provides all peripherals required by the design, and the board is widely available in academic laboratories.

- **HC-SR04.** The sensor is inexpensive, readily available and simple to interface. Unlike a passive infrared sensor, it detects stationary occupants, which is essential in a classroom where occupants remain seated for long periods. Its narrow beam is suited to monitoring a defined zone.
- **LM393 LDR module.** The lighting decision requires only a binary indication of ambient light level, which the module provides directly with a threshold adjustable by its on-board trimmer. No ADC channel or software calibration is therefore required.
- **DHT11.** Provides adequate resolution for display of ambient conditions at minimal cost.
- **SH1106 OLED.** Eight lines of 21 characters are sufficient to present the complete system state; the module requires only a two-wire I2C connection and offers a wide viewing angle.
- **RC522.** The most widely available low-cost 13.56 MHz reader; it reads the unique identifier of MIFARE Classic cards and tags, which are inexpensive to issue to students and staff [10].
- **IRLZ44N, 1N4007 and type-130 motor.** A logic-level MOSFET is the simplest means of switching an inductive load from a 3.3 V output, the 1N4007 is a standard freewheeling diode, and the small DC motor serves as a laboratory representation of a fan.

## 2.5 Control Algorithm

### *2.5.1 Main loop*

The firmware does not use an operating system or task scheduler. Following a one-time initialisation, the main loop executes the following sequence at approximately four iterations per second:

1. Trigger the zone A sensor and wait 60 ms; trigger the zone B sensor and wait 60 ms. The interval allows the echo of one sensor to decay before the other is triggered. The RFID reader and the serial port are serviced during both intervals.
2. At intervals of 2 s, read the DHT11, re-initialise the OLED if it was previously absent, and verify the presence of the RFID reader.
3. Execute any complete command received on the serial interface.
4. Apply button presses recorded by the interrupt handler to the override state.
5. Sample both potentiometers.
6. For each zone, filter the measured distance into an occupancy flag and set the zone light accordingly.
7. Set the fan output from the override state, the RFID check-in count and the speed setting.
8. Clear the check-in roster if credentials remain checked in while both zones have been unoccupied for 2 minutes.
9. Update the eight display lines, servicing the RFID reader between groups of lines.

10. Transmit one status line on the serial interface, except within 5 s of keyboard input.
11. Toggle LD2 as a heartbeat indication.

Each iteration requires approximately 0.25 s. All waits on external hardware within the loop are time-limited, so that a disconnected device results in the loss of a single reading rather than a halt of the system.

### *2.5.2 Occupancy detection*

The HC-SR04 is triggered by a 10 µs pulse, after which it holds its echo output high for the round-trip time of a 40 kHz ultrasonic burst [11]. Taking the speed of sound as approximately 343 m/s, the distance is given by

$$d\ (\text{cm}) = t_{\text{echo}}\ (\mu\text{s}) \times 0.0343 / 2 \approx t_{\text{echo}} / 58 \tag{2.1}$$

A distance below 100 cm is interpreted as an occupant in the corresponding zone. The driver returns one of three results: a measured distance; `DIST_FAR` when the echo exceeds 25 ms, corresponding to no object within approximately 4 m; or `DIST_NO_ECHO` when the sensor does not respond. The lighting logic treats both error results as an unoccupied zone, but the display reports them separately, which enabled the fault in the third zone to be diagnosed.

### *2.5.3 Occupancy filtering*

Individual readings are subject to transient errors caused by occupant movement, missed echoes and objects located near the 100 cm threshold, which would otherwise cause the zone light to flicker. For each zone the firmware therefore maintains both the instantaneous reading and a filtered occupancy flag. The flag changes state only after the instantaneous reading has differed from it continuously for 5 s, for a transition to occupied, or for 10 s, for a transition to unoccupied. A single agreeing reading resets the timer (Fig. 2.5). The longer deactivation delay follows the finding of Guo et al. [2] that premature switch-off while occupants are stationary is more objectionable than a short delay at switch-on.

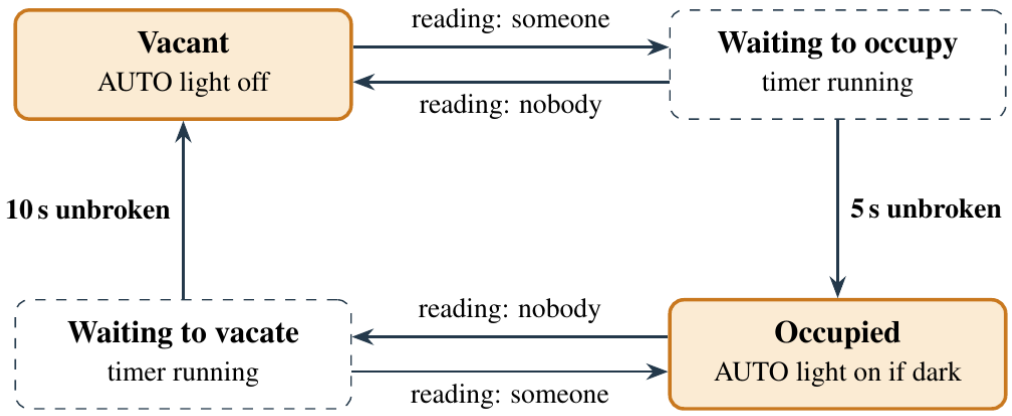



**Fig. 2.5:** State diagram of the per-zone occupancy filter

In the implementation, the two transitional states are represented by a non-zero start time:

```
bool raw = (cm[z] != DIST_NO_ECHO && cm[z] != DIST_FAR && cm[z] < OCC_CM);
if (raw != occ[z]) {
  if (pending_since[z] == 0) pending_since[z] = micros();  /* start timer  */
  uint32_t need_us = (raw ? OCC_MS : VACANT_MS) * 1000UL;  /* 5 s or 10 s  */
  if (micros() - pending_since[z] >= need_us) {
    occ[z] = raw;                                          /* accept       */
    pending_since[z] = 0;
  }
} else {
  pending_since[z] = 0;                                    /* streak broken */
}
```

### *2.5.4 Lighting control*

The operating mode of each zone is defined at compile time. The manual override takes precedence over all modes, and the light potentiometer sets the intensity of an enabled light, as summarised in Table 2.3.

**Table 2.3:** Light output for each override state and mode

| Override | Mode | Light output |
|---|---|---|
| ON | any | Enabled at the set intensity; sensors ignored |
| OFF | any | Disabled |
| AUTO | 0 (OFF) | Disabled |
| AUTO | 1 (AUTO) | Enabled at the set intensity only if the zone is occupied (or the entry boost is active) and ambient light is low |
| AUTO | 2 (DIM) | Approximately 10 %, not exceeding the set intensity |
| AUTO | 3 (MAX) | Enabled at the set intensity |

### *2.5.5 Entrance check-in using RFID*

Each card presentation is processed by the decision sequence shown in Fig. 2.6. The roster accommodates up to eight credentials. A credential presented for the first time is checked in, and each subsequent presentation toggles it between the checked-in and checked-out states. A credential is ignored for 3 s after each detection, and this interval is restarted while the credential remains within the field, so that a card resting on the reader does not toggle repeatedly. An allow list defined in the firmware may restrict the accepted credentials; by default all credentials are accepted.

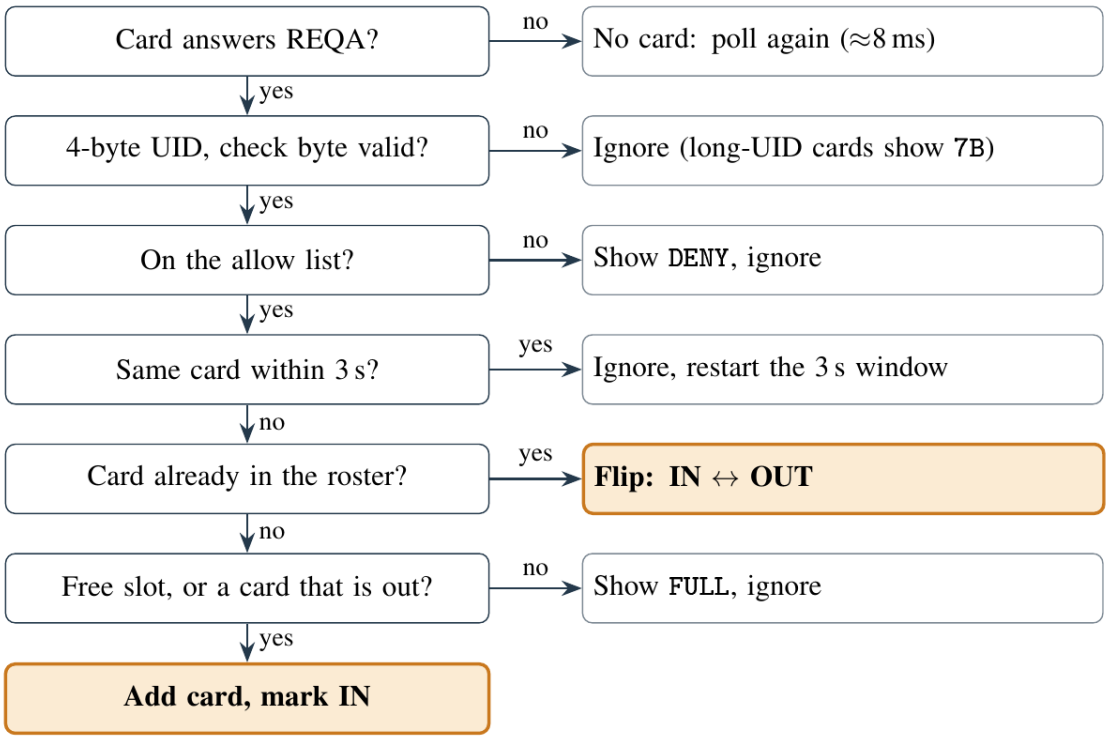



**Fig. 2.6:** Processing of a single card presentation; highlighted outcomes modify the check-in count

A check-in has two effects. Both zones are treated as occupied for 30 s (the entry boost), so that lighting is enabled on entry rather than after the 5 s filter delay, and the fan is enabled while the check-in count is non-zero. Both effects end when the last credential is checked out. To account for occupants who leave without checking out, the roster is cleared when credentials remain checked in but both zones have been unoccupied for 2 minutes.

It should be noted that the RFID subsystem counts presented credentials rather than persons. An occupant who enters without presenting a card is not registered, and a missed presentation reverses the interpretation of the next presentation of that card. For this reason, the zone lights are governed by the ultrasonic occupancy flags rather than by the check-in count.

### *2.5.6 Fan control and manual override*

The fan operates at the speed set by its potentiometer when the override is ON, or when the override is AUTO and at least one credential is checked in; otherwise it is stopped. The lowest 5 % of potentiometer travel corresponds to the off state, and the remaining travel is mapped to 35–100 % of the permitted speed. The 35 % lower limit is imposed because the motor stalls below approximately one third of its rated voltage. Temperature-based fan control with a 26 °C threshold was evaluated initially but was replaced by manual speed control gated by the check-in count, since at the prevailing laboratory temperature of 23 °C the fan would not operate during demonstration.

Each press of the on-board push button B1 advances a common override state through AUTO, ON and OFF for both the lights and the fan. Every change of state is logged with a timestamp.

## 2.6 Firmware Design

The firmware is partitioned according to hardware dependence. The source file `main.c` contains all peripheral drivers, both interrupt handlers, the initialisation routine and the main loop. The header `logic.h` contains all decision logic that can be expressed on integer data: the RFID roster, the potentiometer transfer functions, the lighting and fan rules, the override logic and the date parsing routines. Time is supplied to these functions as an argument, which allows them to be tested on a host computer with synthetic timestamps. The vendor start-up file, linker script and CMSIS register definitions are used without modification. The complete source code is reproduced in Annexure B.

### *2.6.1 Clock, GPIO and timebase*

The microcontroller is clocked from its internal 16 MHz HSI oscillator with the PLL disabled, so that the core and both peripheral buses operate at 16 MHz; all timer and baud-rate constants are derived from this frequency. The clock of each peripheral is enabled in the RCC before configuration, since a peripheral with its clock disabled ignores register writes [7].

Each pin is represented by a port pointer and a pin number. Helper functions configure the mode and pull-up fields in `MODER` and `PUPDR`, the output type in `OTYPER` and the alternate-function field in `AFR`, using read-modify-write operations that preserve the configuration of the remaining pins of the port. Outputs are driven through `BSRR`, which sets or clears a pin in a single atomic write.

TIM2, a 32-bit timer, is configured with a prescaler of 16 to count at 1 MHz and is allowed to overflow freely, so that its counter provides a timestamp with 1 µs resolution. All intervals are evaluated as `micros() - start` in unsigned 32-bit arithmetic, which remains valid across the counter overflow occurring every 2^32 µs (approximately 71.6 minutes). This timebase is used for echo and DHT11 timing, all timeouts, the occupancy filter, the RFID lockout interval, the fan start-up pulse and the button debounce.

### *2.6.2 DHT11 driver*

The DHT11 communicates over a single bidirectional data line, and the corresponding pin is therefore configured as open-drain with the internal pull-up enabled [12]. A measurement is initiated by holding the line low for 20 ms. The sensor responds with a low pulse and a high pulse of 80 µs each, followed by 40 data bits. Each bit begins with a low interval of approximately 50 µs, followed by a high pulse of 26–28 µs for a logic 0 or approximately 70 µs for a logic 1; a high pulse exceeding 45 µs is decoded as a 1. The five received bytes contain the integral and decimal parts of humidity and temperature and a checksum equal to the least significant byte of their sum. Each edge is awaited with a 100 µs timeout, and the sensor is read at intervals of 2 s.

### *2.6.3 I2C driver and SH1106 OLED*

I2C1 is operated in standard mode at 100 kHz. The timing registers are derived from the 16 MHz bus clock as follows:

$$\text{CR2} = 16, \quad \text{CCR} = 16\ \text{MHz} / (2 \times 100\ \text{kHz}) = 80, \quad \text{TRISE} = 1000\ \text{ns} / 62.5\ \text{ns} + 1 = 17 \tag{2.2}$$

Each write follows the standard master transmitter sequence: generation of a START condition and wait for `SB`; transmission of the address and wait for `ADDR`; clearing of `ADDR` by reading `SR1` and `SR2`; transmission of a control byte followed by the data, with a wait for `TXE` before each byte and for `BTF` after the last; and generation of a STOP condition. Every wait is limited to 2 ms and the acknowledge-failure flag is checked, so that an absent display cannot halt the main loop. Before the peripheral is enabled, the firmware applies nine manual clock pulses to SCL followed by a STOP condition, which releases a slave device left holding SDA low by a reset during a transfer.

The SH1106 module may be configured for address 0x3C or 0x3D, and the driver probes both [13]. The display memory is organised as eight pages of eight pixel rows, each of which holds one line of 21 characters in a 5×7 font. The controller provides 132 columns of display memory, of which columns 2 to 129 are visible; each line is therefore written with a column offset of 2. In the event of a write failure, the display is marked as absent and re-initialisation is attempted every 2 s, allowing the module to be reconnected without a system reset.

### *2.6.4 PWM generation for lighting*

TIM3 and TIM4 generate the PWM signals for the zone lights at a frequency of

$$f_PWM = 16\ \text{MHz} / ((\text{PSC} + 1)(\text{ARR} + 1)) = 16\ \text{MHz} / (16 \times 1000) = 1\ \text{kHz} \tag{2.3}$$

In PWM mode 1 the output is high while the counter value is below the compare register, giving a duty cycle of `CCR`/1000. Register preload is enabled so that a new duty cycle takes effect at the start of the following period, preventing truncated pulses.

### *2.6.5 ADC and potentiometer inputs*

ADC1 performs single conversions with a sampling time of 144 cycles, which allows the sampling capacitor to charge fully through the 10 kΩ source impedance of the potentiometer. Each reading is the average of eight conversions. The light potentiometer is mapped from the range 0–4095 to PWM values of 50–999 (5–100 %); a zero output is not produced, since disabling the light is the function of the operating mode and the override. Small variations are ignored in both transfer functions to prevent ADC noise from causing visible flicker or speed fluctuation.

### *2.6.6 SPI driver and RC522*

SPI2 is operated as master in mode 0 at 16 MHz / 16 = 1 MHz, which is well within the rating of the RC522 and suitable for breadboard wiring. Chip select is controlled in software on PB12,

and each register access consists of an address byte, with bit 7 set for a read operation, followed by a data byte [10].

During initialisation the reader is reset through PC5 and by a software reset command, and its version register is read; a value of 0x00 or 0xFF indicates that the device is not responding. The internal timer of the RC522 is configured to terminate a transaction after 8 ms without a response, which limits the duration of a poll when no card is present. Card identification then follows the two-step procedure defined in ISO/IEC 14443-3 [14]. A REQA command (0x26, transmitted as 7 bits) is answered by any card in the field with a two-byte response, after which an anticollision command (0x93 0x20) returns the four UID bytes and a check byte equal to their exclusive-OR. A card that has completed anticollision remains in the ready state and does not respond to further REQA commands until it leaves the field; consequently, a card held on the reader is registered only once.

### *2.6.7 Fan PWM generation*

TIM1 drives the MOSFET gate at a frequency of

$$f = 16\ \text{MHz} / ((0 + 1)(799 + 1)) = 20\ \text{kHz} \tag{2.4}$$

A frequency of 20 kHz was selected because operation at 1 kHz produces audible noise from the motor. TIM1 is an advanced-control timer, and its main output enable bit `MOE` must be set for the output to be active. Since the motor is rated for 3–6 V and the supply is 9 V, the duty cycle corresponding to full speed is limited to 60 %, giving an average motor voltage of

$$V_{avg} \approx D \times V_{bat} = 0.6 \times 9\ \text{V} = 5.4\ \text{V} \tag{2.5}$$

As a stationary motor requires greater torque to start than to continue running, full permitted duty cycle is applied for 0.3 s whenever the commanded speed changes from zero to a non-zero value, after which the requested speed is applied.

### *2.6.8 Real-time clock*

The RTC is clocked from the 32.768 kHz crystal fitted on the Nucleo board. Since 32 768 = 2^15, asynchronous and synchronous prescalers of 128 and 256 yield a 1 Hz time base. The RTC resides in the backup domain, which is not affected by a system reset; modification of its registers requires enabling the power interface clock, setting the `DBP` bit, writing the key sequence 0xCA and 0x53 to the write-protection register, and entering initialisation mode. Time and date are stored in BCD format.

At start-up the firmware compares a hash of the compile date and time with a value held in the backup register `BKP0R`. If the values match, the device has only been reset and the running clock is retained; otherwise, the clock is initialised from the compile time. The time may be set precisely over the serial interface using commands such as `T 08:30:00` and `D 21/10/26`. If the crystal oscillator fails to start within 2.5 s, the internal LSI oscillator is used and this condition is indicated on the display.

### *2.6.9 Interrupt handling*

Two interrupt sources are used. In both cases the interrupt handler performs only the minimum necessary processing, and the associated work is carried out in the main loop [15].

**Push button.** B1 is connected to PC13 with an external pull-up resistor, so that a press produces a falling edge. EXTI line 13 is mapped to port C through `SYSCFG->EXTICR[3]`, configured for falling-edge triggering, unmasked, and enabled in the NVIC under the shared `EXTI15_10` vector. The handler clears the pending flag, rejects edges occurring within 250 ms of the previous accepted press to suppress contact bounce, and increments a counter that is read by the main loop once per iteration.

```
void EXTI15_10_IRQHandler(void) {
  if (EXTI->PR & (1U << 13)) {
    EXTI->PR = (1U << 13);                  /* write 1 to clear */
    uint32_t now = TIM2->CNT;
    if (now - btn_last_us >= BTN_DEBOUNCE_US) {
      btn_presses++;
      btn_last_us = now;
    }
  }
}
```

**Serial reception.** Characters may arrive while the main loop is occupied with sensor measurements, and polled reception would result in data loss. With `RXNEIE` enabled, each received byte generates a USART2 interrupt; the handler reads the data register, which clears the receive and overrun flags, and stores the byte in a 64-byte circular buffer. Since only the handler advances the head index and only the main loop advances the tail index, no further synchronisation is required. Variables shared with interrupt handlers are declared `volatile` so that the compiler does not retain stale copies in registers. The handlers are kept short so that no I2C or SPI transaction is initiated from interrupt context.

## 2.7 Tools Used

**Table 2.4:** Software and hardware tools

| Tool | Purpose |
|---|---|
| GNU Arm Embedded Toolchain (arm-none-eabi-gcc) | Compilation and linking of the firmware |
| CMake and Ninja | Build configuration |
| STM32CubeProgrammer CLI | Programming over SWD with read-back verification |
| Visual Studio Code | Source code editing |
| Serial terminal (115 200 baud) | Status monitoring and time-setting commands |
| Wokwi simulator | Initial verification of control logic on a simulated Bluepill |
| Arduino IDE with STM32duino | Reference implementation used for wiring verification |
| GCC on host computer | Execution of the 460 logic test cases |
| Digital multimeter | Verification of supply voltages, continuity and pin levels |
| Reference documents | Reference manual, board manual and component datasheets listed under References |

## 2.8 Preliminary Results

At the mid-semester review, the two occupancy zones, the four lighting modes, the LDR input, the DHT11 sensor, the OLED display and the occupancy filter were operational on hardware, with a program size of approximately 6.8 KB. The serial status output and heartbeat indication had by then enabled the diagnosis of the faults described in Section 3.5. The RFID reader, potentiometers, RTC, interrupt handling and fan control were added subsequently.

## 2.9 Conclusions

The methodology is based on three decisions: the use of a development platform with integrated debugging and serial output, the separation of wiring verification from firmware verification, and a firmware structure in which every hardware access is time-bounded and all decision logic can be tested on a host computer. On this basis, each subsystem was developed and verified individually before integration into the main loop described above.

# CHAPTER 3

# RESULT ANALYSIS

## 3.1 Introduction

This chapter presents the results obtained with the completed system. Photographs of the hardware and the display are followed by the functional test results, an analysis of the occupancy filter supported by a graphical representation, the memory utilisation and loop timing, and a discussion of the deviations encountered during development together with their resolution.

## 3.2 Photographs of the Working Project

The complete system is shown in Fig. 3.1, and the display during normal operation in Fig. 3.2. The content of each display line is described in Table 3.1.

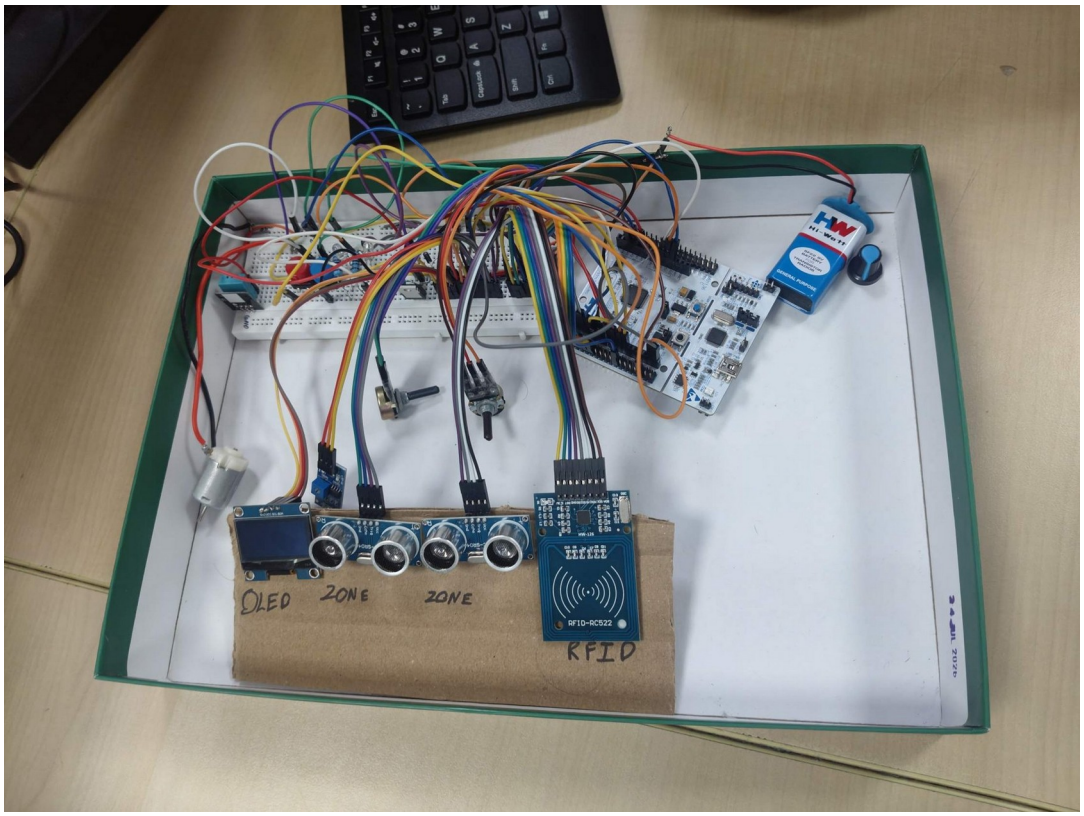



**Fig. 3.1:** Complete system comprising the Nucleo-F401RE, zone sensors and lights, DHT11, LDR module, OLED display, RC522 reader, potentiometers and motor driver




**Fig. 3.2:** OLED display layout during normal operation, rendered from the firmware font and line formats

**Table 3.1:** OLED display layout (one text line per 8-pixel page)

| Line | Example | Content |
|---|---|---|
| 1 | `LIGHT: DARK` | Ambient light state from the LDR module |
| 2 | `IN:2 3A4B5C6D IN` | Check-in count, UID of the last card and the resulting action |
| 3 | `A: 45cm OCC AUT` | Zone A distance, filtered occupancy and operating mode |
| 4 | `B:FAR AUT` | Zone B, in the same format |
| 5 | `21/10 08:30:12` | Date and time from the RTC |
| 6 | `T:30C H:65%` | Temperature and humidity from the DHT11 |
| 7 | `OCC1/2 AUTO L75 F60` | Occupied zones, override state, light intensity (%), fan speed (%) |
| 8 | `RFID OK V92` | RFID reader status and chip version |

## 3.3 Result Analysis

### *3.3.1 Functional tests*

The tests performed on the assembled hardware, the expected behaviour in each case and the outcome are listed in Table 3.2.

**Table 3.2:** Functional test results

| Test | Expected behaviour | Result |
|---|---|---|
| Programming | Download verified; device restarts | Pass |
| Heartbeat | LD2 toggles approximately four times per second | Pass |
| Start-up report | Serial output reports RTC time, OLED address and RC522 version | Pass |
| Zone A / zone B occupancy | Occupied state indicated 5 s after an object is placed within 100 cm and cleared 10 s after its removal | Pass |
| Ambient-light gating | Light of an occupied zone enabled only when the LDR indicates low light | Pass |
| Light intensity control | Displayed intensity and LED brightness follow the potentiometer over 5–100 % | Pass |

| Test | Expected behaviour | Result |
|---|---|---|
| RFID check-in | Check-in indicated, count incremented, both lights enabled for 30 s under low light | Pass |
| Fan start | Fan starts with an initial full-duty pulse after check-in; speed follows the potentiometer | Pass |
| RFID check-out | Repeated presentation after 3 s indicates check-out; fan stops when the count reaches zero | Pass |
| Manual override | Button cycles through ON, OFF and AUTO; each change logged | Pass |
| Clock setting | `T 08:30:00` received over serial updates the clock | Pass |
| Fault tolerance | Disconnection of a sensor or the OLED does not interrupt the heartbeat or serial output | Pass |

### *3.3.2 Occupancy filter response*

Fig. 3.3 shows the response of the occupancy filter to a scripted input sequence in which an occupant arrives at 0.25 s, a single echo is missed at 2 s, the occupant leaves at 20 s and returns briefly at 25 s. The sequence was processed by the same algorithm as the firmware, with one sample per loop iteration of 0.25 s. The corresponding sequence of states is listed in Table 3.3.

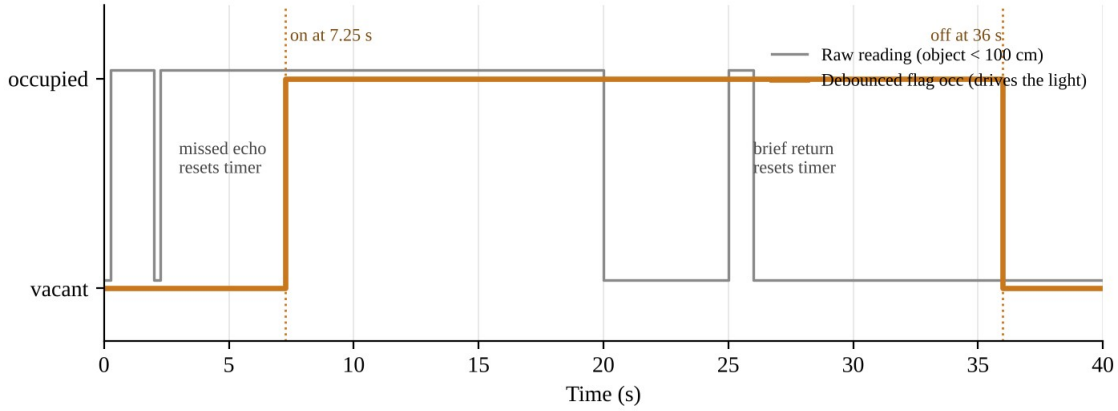



**Fig. 3.3:** Instantaneous occupancy reading and filtered occupancy flag for one zone

**Table 3.3:** Occupancy filter state sequence for one zone (one iteration ≈ 0.25 s)

| Time | Raw | Event | Flag |
|---|---|---|---|
| 0.00 s | no | Zone unoccupied | vacant |
| 0.25 s | yes | Occupant arrives; timer started | vacant |
| 2.00 s | no | Missed echo; timer reset | vacant |
| 2.25 s | yes | Timer restarted | vacant |
| 7.25 s | yes | 5 s elapsed without interruption | **occupied** |
| 20.0 s | no | Occupant leaves; timer started | occupied |
| 25.0 s | yes | Occupant returns; timer reset | occupied |
| 26.0 s | no | Timer restarted | occupied |
| 36.0 s | no | 10 s elapsed without interruption | **vacant** |

The results confirm the intended behaviour of the filter. Both the missed echo at 2 s and the brief return at 25 s reset the timer, and neither transient affects the light. This immunity is obtained at the cost of additional delay: the light is enabled 7 s after arrival rather than 5 s, because the missed echo restarted the timer, and is disabled 16 s after the occupant first leaves, because the return restarted the 10 s interval. For a classroom application, a delay of a few seconds is preferable to visible flicker of the lighting.

### *3.3.3 Memory utilisation and timing*

**Table 3.4:** Memory utilisation and loop timing

| Parameter | Value |
|---|---|
| Program memory (code and constants) | Approximately 17 KB of 512 KB |
| Data memory (data and bss) | Approximately 1.8 KB of 96 KB |
| Arduino reference implementation at mid-semester (fewer features) | Approximately 42 KB of program memory |
| Main loop iteration | Approximately 0.25 s |
| Ultrasonic measurements and intervals | Approximately 125 ms per iteration |
| Display update, 8 lines at 100 kHz | Approximately 60–70 ms per iteration |
| RFID poll with no card present | Approximately 8 ms |
| Host-based logic tests | 460 test cases, all passed |

The bare-metal implementation requires less than half the program memory of the earlier Arduino implementation while providing considerably more functionality, since it contains only the register operations required by the application. The loop period is dominated by the ultrasonic measurements, whose 60 ms intervals cannot be reduced without cross-talk between zones, and by the display update. An iteration period of 0.25 s corresponds to a display refresh rate of four updates per second, which satisfies the specification.

## 3.4 Significance of the Results

All specifications listed in Table 1.1 were met. Lighting is enabled only in occupied zones under low ambient light, the fan operates only while at least one credential is checked in, both loads respond within the designed delays, and operation continues when any device is disconnected. These results demonstrate that zone-level control combined with entrance check-in can be implemented with low-cost modules on a single microcontroller, without cameras, network connectivity or vendor libraries, which makes the approach practical for institutional classrooms.

## 3.5 Deviations from Expected Results and Justification

Several deviations from the expected behaviour were encountered during development. These are listed in Table 3.5 in the order in which they occurred, and the more significant cases are discussed below.

**Table 3.5:** Deviations encountered and their resolution

| Deviation | Cause | Resolution |
|---|---|---|
| No output on the Blackpill OLED | No debugger or serial interface available for diagnosis | Development moved to the Nucleo-F401RE |
| Wiring and firmware faults indistinguishable | Wiring and firmware introduced simultaneously | Wiring verified first with an Arduino reference implementation |
| System halted when the OLED was disconnected | Unbounded waits on I2C status flags | Timeout of 2 ms on every wait; bus recovery at start-up |
| Programming reported success but previous firmware continued to run | Unreliable mass-storage programming with the 2019 ST-LINK firmware | Programming over SWD with the STM32CubeProgrammer CLI |
| OpenOCD unable to open the debug probe | Same ST-LINK firmware version | STM32CubeProgrammer CLI used instead |
| LDR input permanently indicating darkness | Module output is high under low light, contrary to the initial assumption | Polarity verified on the module; pull-up enabled on PB0 |
| Third zone permanently reporting no echo | Wiring fault; identical code operated correctly for zones A and B | Fault located through distinct error codes; zone removed |
| Light flicker near the 100 cm threshold | Isolated readings crossing the threshold | Asymmetric 5 s / 10 s occupancy filter |
| Zone B light inoperative while zone A operated | Fault associated with the D10 connection rather than the LED | Located by exchanging connections; connection replaced |
| OLED frozen on a previous frame after addition of the light potentiometer | Potentiometer placed near the I2C wiring, display left in an incomplete transfer, or both | Potentiometer relocated; system power-cycled |
| Temperature-controlled fan inactive at 23 °C | 26 °C threshold above laboratory temperature (operation as designed) | Manual speed control gated by check-in count |

### *3.5.1 System halt on I2C communication failure*

The initial I2C driver polled the peripheral status flags in unbounded loops. When the display did not acknowledge, these loops did not terminate and the entire system appeared inoperative. Every wait was subsequently limited to 2 ms with an additional check of the acknowledge-failure flag, and the same principle of bounding every hardware wait was applied to all other drivers.

### *3.5.2 Fault in the third zone*

Fig. 3.4 shows an early serial log obtained from the Arduino reference implementation. All three zones report NOECHO at this stage, since the wiring had not yet been completed. In subsequent sessions zones A and B became operational, whereas zone C continued to report NOECHO after replacement of the sensor. Because the firmware distinguishes a non-responding sensor from one that detects no object, it was established that the sensor was not being received at all, indicating a wiring fault. This finding led to the adoption of a two-zone configuration.

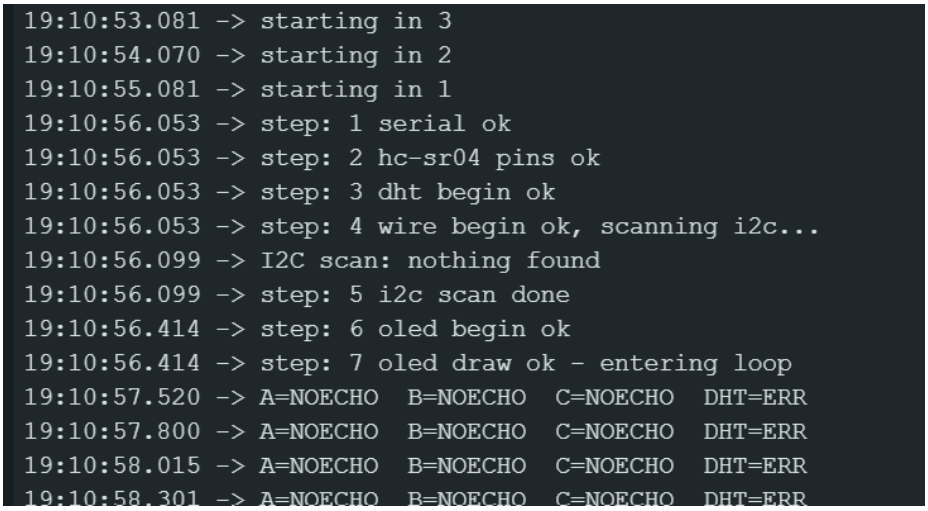



**Fig. 3.4:** Early start-up log received over the ST-LINK virtual COM port

### *3.5.3 Display failure after addition of the light potentiometer*

Following the addition of the light potentiometer, the display ceased to update (Fig. 3.5). It continued to show the last frame produced by the previous firmware version, including the superseded “OCCUPIED: 1/2” line and no clock line, together with two solid blocks resulting from interrupted writes. The serial output and heartbeat confirmed that the new firmware was executing, which located the fault in the I2C path; the new code did not access I2C1 or its pins. Normal operation was restored after the potentiometer was removed and the system was power-cycled. As both changes were made together, the individual cause cannot be identified with certainty. The potentiometer had been placed on breadboard rows adjacent to the display wiring, and a display left in an incomplete transfer remains in that state until power is removed, which a system reset does not achieve. The potentiometer was relocated to separate rows.

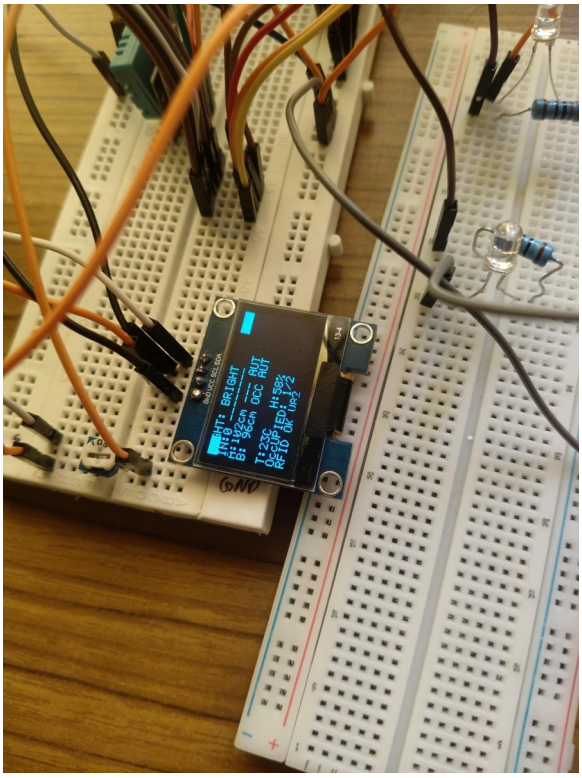


**Fig. 3.5:** Display retaining the frame of the previous firmware; the trimmer potentiometer is visible at the lower left

### *3.5.4 Unreliable programming*

The ST-LINK of the Nucleo board was supplied with firmware dating from 2019. Programming by copying the binary to the mass-storage device occasionally reported success while the previous program continued to execute, a condition that is indistinguishable from a firmware defect. Programming over SWD with the STM32CubeProgrammer command-line tool, invoked through a batch script that builds, programs, verifies and resets the device, proved reliable for the remainder of the project.

## 3.6 Conclusions

The system meets its specifications on hardware, and the occupancy filter performs as designed, rejecting single-sample errors at the expense of a delay of a few seconds. The majority of deviations during development arose from wiring and tooling rather than from the control design, and each was resolved through systematic fault isolation supported by the diagnostic outputs of the system.

# CHAPTER 4

# CONCLUSION AND FUTURE SCOPE OF WORK

## 4.1 Work Conclusion

Lighting and ventilation in conventional classrooms are controlled for the room as a whole and frequently remain energised over unoccupied areas. The objective of this project was a low-cost controller that enables the light of each zone only under occupancy and low ambient light, operates the fan only while occupants are checked in, and continues to function when individual components fail.

The classroom was divided into two zones, each equipped with an ultrasonic sensor and an independent light. Occupancy was filtered with asymmetric activation and deactivation delays, and an RFID reader at the entrance maintained a check-in count used to enable the fan and to energise the lighting on entry. Potentiometers provide adjustment of light intensity and fan speed, a push button serviced by an external interrupt provides manual override, and the on-chip RTC timestamps all events. The firmware was implemented in bare-metal C, with all decision logic separated from hardware access and verified on a host computer.

The completed system met every target specification. It employs eight on-chip peripherals and two interrupt sources within approximately 17 KB of program memory, less than half that of an earlier Arduino implementation with fewer features. The work shows that effective classroom automation does not require cameras, cloud services or costly controllers. The development further highlighted the value of a debuggable hardware platform, time-bounded hardware access, distinct reporting of each fault condition and the modification of one variable at a time during fault diagnosis.

## 4.2 Future Scope of Work

The most significant improvement would be the addition of a second reader or a pair of break-beam sensors at the entrance, allowing entries and exits to be distinguished so that the check-in count no longer depends on occupants checking out; this would resolve the principal limitations of the present RFID scheme. A passive infrared sensor alongside each ultrasonic sensor would extend coverage to occupants outside the narrow ultrasonic beam. A DS3231 real-time clock module with battery backup would retain time across power interruptions, a SysTick-based scheduler would decouple the ultrasonic measurements, display update and RFID polling, and the independent watchdog would provide recovery from any fault not already covered by a timeout. Finally, the LEDs and the 9 V battery represent the actual loads; a deployable unit would require a relay or solid-state switching stage for mains-powered lighting and fans, designed and verified by qualified personnel, and would be built around the STM32F401 device rather than a development board.

# REFERENCES


[1] D. B. Floyd, D. S. Parker and J. R. Sherwin, “Measured field performance and energy savings of occupancy sensors: three case studies,” in *Proc. ACEEE Summer Study on Energy Efficiency in Buildings*, 1996, pp. 4.97–4.105.

[2] X. Guo, D. K. Tiller, G. P. Henze and C. E. Waters, “The performance of occupancy-based lighting control systems: a review,” *Lighting Research and Technology*, vol. 42, no. 4, pp. 415–431, 2010, doi: 10.1177/1477153510376225.

[3] R. Ani, S. Krishna et al., “An approach towards building an IoT based smart classroom,” in *Proc. Int. Conf. on Advances in Computing, Communications and Informatics (ICACCI)*, 2018.

[4] A. B. Ibrahim, N. Shamsudin, M. H. Ab. Majid, S. Dzulkifly, Y. Abdullah and M. I. Setiawan, “Smart classroom: a development of electrical control system with Internet of Things,” *Journal of ICT in Education*, vol. 11, no. 1, 2024, doi: 10.37134/jictie.vol11.1.8.2024.

[5] D. Mijic and O. Bjelica, “An RFID-based system for efficient classroom management and integration into quality assurance system,” in *ICT Innovations 2013 Web Proceedings*, 2013, p. 208.

[6] STMicroelectronics, *UM1724 User manual: STM32 Nucleo-64 boards (MB1136)*.

[7] STMicroelectronics, *RM0368 Reference manual: STM32F401xB/C and STM32F401xD/E advanced Arm-based 32-bit MCUs*.

[8] STMicroelectronics, *STM32F401xD, STM32F401xE datasheet*.

[9] International Rectifier (Infineon), *IRLZ44N HEXFET Power MOSFET datasheet*.

[10] NXP Semiconductors, *MFRC522 Standard performance MIFARE and NTAG frontend*, product data sheet.

[11] ElecFreaks, *HC-SR04 Ultrasonic Ranging Module user guide*.

[12] Aosong Electronics, *DHT11 Humidity and Temperature Sensor datasheet*.

[13] Sino Wealth, *SH1106: 132 x 64 dot matrix OLED/PLED segment/common driver with controller*, datasheet.

[14] ISO/IEC 14443-3, *Identification cards – Contactless integrated circuit cards – Proximity cards – Part 3: Initialization and anticollision*.

[15] Arm Ltd., *Cortex-M4 Devices Generic User Guide*.

# ANNEXURES

## Annexure A  Serial Interface

The board enumerates as an ST-LINK virtual COM port operating at 115 200 baud. After reset, the results of initialisation are reported, followed by one status line per iteration of the main loop; every card presentation and button press is logged with a timestamp:

```
[08:30:12] A=45(OCC) AUT  B=FAR      AUT  LDR=DARK  T=30 H=65  IN=2(boost)
           RFID=ok  OVR=AUTO BRT=75% FAN=60%  OLED=ok
[08:30:14] RFID 3A4B5C6D IN  inside=2
[08:31:02] Button: override ON
```

The following commands are accepted: `T 08:30:00` or `T 08:30` sets the time; `D 21/10/26` or `D 21/10/2026` sets the date; `?` reports the current date and time.

## Annexure B  Firmware Source: main.c

Complete firmware source as used in the final demonstration. The 5×7 font table (`font_table.inc`) and the vendor start-up, linker and CMSIS files are not reproduced.

### main.c

```
/*
 * Smart Classroom, Nucleo-F401RE, bare metal CMSIS, no HAL.
 * 2 zones: HC-SR04 occupancy + class-light LED each, shared DHT11 + OLED,
 * LM393 module for the AUTO mode's darkness check, RC522 RFID reader at the
 * door, one pot dimming both LEDs, on-chip RTC for the clock, one fan motor
 * per zone.
 *
 * Zone A  TRIG PA10 (D2)   ECHO PB3 (D3)   LED PC7 (D9)   TIM3_CH2
 * Zone B  TRIG PB5  (D4)   ECHO PB4 (D5)   LED PB6 (D10)  TIM4_CH1
 * DHT11   PA9 (D8)
 * OLED    SCL PB8 (D15)  SDA PB9 (D14), I2C1
 * LDR     PB0 (A3), LM393 DO pin, HIGH = dark on this module
 * RC522   SPI2: SCK PB13  MISO PB14  MOSI PB15  SDA(SS) PB12  RST PC5   3V3 ONLY
 * POT     light: wiper on PA0 (A0), ADC1 ch0.  fan: wiper on PA4 (A2), ADC1 ch4.
 *         Pot ends on 3V3 and GND.
 * FAN     PA8 (D7)  TIM1_CH1 -> 220R -> IRLZ44N gate (10k gate-GND), 20 kHz PWM.
 *         Motor on the 9 V battery, 1N4007 across it, battery (-) to Nucleo GND.
 * BUTTON  B1 (blue, on the board), PC13, EXTI line 13 interrupt
 * HC-SR04 on 5V, everything else 3V3, grounds common.
 *
 * Each zone's mode (OFF/AUTO/DIM/MAX) is a #define below; the pot sets how
 * bright "on" is, the blue button cycles a manual override AUTO -> ON -> OFF
 * for lights and fan together. The fan runs while RFID says someone is
 * inside, at the speed set by the fan pot, capped at FAN_MAX_PCT of the battery.
 * Occupancy is debounced: needs OCC_MS of continuous detection to go
 * occupied, VACANT_MS of continuous clear to go back to empty. Tapping a card
 * at the door toggles that card in/out of the headcount; a tap-in also makes
 * both zones count as occupied for ENTRY_BOOST_MS.
 *
 * Clock: on-chip RTC, 32.768 kHz LSE crystal (falls back to the rough LSI if
 * the crystal is missing). Set from the build time on flashing; set it exactly
 * over the serial port with "T 08:30:00" and "D 18/10/26".
 *
 * Interrupts: EXTI15_10 (button) and USART2 RX (serial commands). Everything
 * else is polled. HSI 16MHz, no PLL. UART2 prints raw state every loop and
 * LD2 toggles as a heartbeat. Pure logic lives in logic.h (host tested).
 */

#include "stm32f4xx.h"
#include <stdint.h>
#include <stdbool.h>
#include "logic.h"

#define ZONE_A_MODE 1   /* 0 off, 1 auto, 2 dim, 3 max */
#define ZONE_B_MODE 1
#define FAN_MAX_PCT 60  /* 9 V battery x 60 % = ~5.4 V average for 3-6 V motors. 100 for 9 V
 motors */
#ifndef USE_POT
#define USE_POT     1   /* 0 until the pot is wired, or A0 floats and brightness wanders */
#endif
#ifndef USE_FAN_POT
#define USE_FAN_POT 1   /* fan speed knob on A2. 0 = fan runs at full allowed speed when on */
#endif

#define OCC_CM      100
#define OCC_MS      5000
#define VACANT_MS   10000
#define DHT_PERIOD_US 2000000
#define PING_GAP_MS 60
#define I2C_TIMEOUT_US 2000

#define ENTRY_BOOST_MS 30000UL       /* after a tap-in, both zones count as occupied */
#define TAP_LOCKOUT_MS 3000UL        /* same card ignored this long after a sighting */
#define STALE_RESET_MS 120000UL      /* headcount > 0 but zones empty this long: clear it */
#define POT_SAMPLES    8             /* ADC reads averaged per pot reading */
#define BTN_DEBOUNCE_US 250000UL     /* ignore button edges closer than this */
#define RX_QUIET_US    5000000UL     /* hold the status line this long after you type */

```

```
#define DIST_NO_ECHO 0xFFFE
#define DIST_FAR     0xFFFF

#define LED_OFF  0
#define LED_DIM  100
#define LED_FULL 999

typedef struct { GPIO_TypeDef *port; uint8_t pin; } pin_t;

static const pin_t TRIG[2] = { {GPIOA, 10}, {GPIOB, 5} };
static const pin_t ECHO[2] = { {GPIOB, 3},  {GPIOB, 4} };
static const pin_t DHT  = {GPIOA, 9};
static const pin_t LED  = {GPIOA, 5};        /* LD2 */
static const pin_t LDR  = {GPIOB, 0};
static const pin_t RC_CS  = {GPIOB, 12};    /* RC522 SDA/SS, software chip select */
static const pin_t RC_RST = {GPIOC, 5};

/* Cards that may open the door. 0 ends the list; {0} alone = accept any card.
 * Tap a card, read its UID off the UART or OLED, paste it here as 0xAABBCCDD. */
static const uint32_t ALLOWED_UIDS[] = { 0 };
static const int   mode[2] = { ZONE_A_MODE, ZONE_B_MODE };
static const char *MODE_TXT[4] = { "OFF", "AUT", "DIM", "MAX" };

static void pin_mode(pin_t p, uint32_t mode) {
  p.port->MODER = (p.port->MODER & ~(3U << (p.pin * 2))) | (mode << (p.pin * 2));
}
static void pin_pull(pin_t p, uint32_t pupd) {
  p.port->PUPDR = (p.port->PUPDR & ~(3U << (p.pin * 2))) | (pupd << (p.pin * 2));
}
static void pin_od(pin_t p, bool od) {
  if (od) p.port->OTYPER |= (1U << p.pin); else p.port->OTYPER &= ~(1U << p.pin);
}
static void pin_af(pin_t p, uint32_t af) {
  uint32_t i = p.pin >> 3, sh = (p.pin & 7) * 4;
  p.port->AFR[i] = (p.port->AFR[i] & ~(0xFU << sh)) | (af << sh);
}
static inline void pin_hi(pin_t p) { p.port->BSRR = (1U << p.pin); }
static inline void pin_lo(pin_t p) { p.port->BSRR = (1U << (p.pin + 16)); }
static inline bool pin_rd(pin_t p) { return (p.port->IDR >> p.pin) & 1U; }

static inline uint32_t micros(void) { return TIM2->CNT; }
static void delay_us(uint32_t us) { uint32_t s = micros(); while (micros() - s < us); }
static void delay_ms(uint32_t ms) { while (ms--) delay_us(1000); }

static void uart_init(void) {
  pin_t tx = {GPIOA, 2};
  pin_af(tx, 7); pin_mode(tx, 2);
  USART2->BRR = 139;
  USART2->CR1 = USART_CR1_TE | USART_CR1_UE;
}
static void uart_putc(char c) { while (!(USART2->SR & USART_SR_TXE)); USART2->DR = (uint8_t)c;
 }
static void uart_puts(const char *s) { while (*s) uart_putc(*s++); }
static void uart_putu(uint32_t v) {
  char b[10]; int i = 0;
  do { b[i++] = '0' + v % 10; v /= 10; } while (v);
  while (i--) uart_putc(b[i]);
}
static void uart_puthex8(uint8_t v) {
  const char *h = "0123456789ABCDEF";
  uart_putc(h[v >> 4]); uart_putc(h[v & 15]);
}

/* ---------- interrupt-fed state ---------- */
/* USART2 RX: the ISR drops each byte into a ring buffer, main() reads it. */
#define RX_SIZE 64
static volatile uint8_t  rx_buf[RX_SIZE];
static volatile uint8_t  rx_head;            /* written only by the ISR  */
static volatile uint8_t  rx_tail;            /* written only by main()   */
static volatile uint32_t rx_last_us;
static volatile bool     rx_any;             /* anything typed since boot */

/* Blue button: the ISR only counts presses; main() decides what they mean. */
static volatile uint32_t btn_presses;
static volatile uint32_t btn_last_us;

void USART2_IRQHandler(void) {
  uint32_t sr = USART2->SR;
  if (sr & (USART_SR_RXNE | USART_SR_ORE)) {
    uint8_t c = (uint8_t)USART2->DR;         /* reading DR clears RXNE and ORE */
    uint8_t next = (uint8_t)((rx_head + 1) % RX_SIZE);
    if (next != rx_tail) { rx_buf[rx_head] = c; rx_head = next; }   /* full: drop */
```

```
    rx_last_us = TIM2->CNT;
    rx_any = true;
  }
}

void EXTI15_10_IRQHandler(void) {
  if (EXTI->PR & (1U << 13)) {
    EXTI->PR = (1U << 13);                  /* write 1 to clear the pending flag */
    uint32_t now = TIM2->CNT;
    if (now - btn_last_us >= BTN_DEBOUNCE_US) { btn_presses++; btn_last_us = now; }
  }
}

static void uart_rx_init(void) {
  pin_t rx = {GPIOA, 3};
  pin_af(rx, 7); pin_mode(rx, 2);
  USART2->CR1 |= USART_CR1_RE | USART_CR1_RXNEIE;
  NVIC_EnableIRQ(USART2_IRQn);
}

static void button_init(void) {
  pin_t b1 = {GPIOC, 13};                   /* board has its own pull-up, active low */
  pin_mode(b1, 0);
  RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
  SYSCFG->EXTICR[3] = (SYSCFG->EXTICR[3] & ~SYSCFG_EXTICR4_EXTI13) | SYSCFG_EXTICR4_EXTI13_PC;
  EXTI->FTSR |= (1U << 13);                 /* falling edge = pressed */
  EXTI->RTSR &= ~(1U << 13);
  EXTI->PR    = (1U << 13);
  EXTI->IMR  |= (1U << 13);
  NVIC_EnableIRQ(EXTI15_10_IRQn);
}

static int uart_getc(void) {
  if (rx_tail == rx_head) return -1;
  int c = rx_buf[rx_tail];
  rx_tail = (uint8_t)((rx_tail + 1) % RX_SIZE);
  return c;
}

static uint16_t read_cm(int z) {
  if (pin_rd(ECHO[z])) return DIST_NO_ECHO;
  pin_hi(TRIG[z]); delay_us(10); pin_lo(TRIG[z]);
  uint32_t s = micros();
  while (!pin_rd(ECHO[z])) if (micros() - s > 5000) return DIST_NO_ECHO;
  uint32_t t0 = micros();
  while (pin_rd(ECHO[z])) if (micros() - t0 > 25000) return DIST_FAR;
  return (uint16_t)((micros() - t0) / 58);
}

static bool wait_level(pin_t p, bool level, uint32_t timeout_us) {
  uint32_t s = micros();
  while (pin_rd(p) != level) if (micros() - s > timeout_us) return false;
  return true;
}

static bool dht11_read(uint8_t *temp_c, uint8_t *hum) {
  uint8_t d[5] = {0};
  pin_lo(DHT); delay_ms(20);
  pin_hi(DHT);
  if (!wait_level(DHT, 0, 100)) return false;
  if (!wait_level(DHT, 1, 100)) return false;
  if (!wait_level(DHT, 0, 100)) return false;
  for (int i = 0; i < 40; i++) {
    if (!wait_level(DHT, 1, 100)) return false;
    uint32_t t = micros();
    if (!wait_level(DHT, 0, 100)) return false;
    if (micros() - t > 45) d[i / 8] |= (uint8_t)(1U << (7 - (i % 8)));
  }
  if ((uint8_t)(d[0] + d[1] + d[2] + d[3]) != d[4]) return false;
  *hum = d[0]; *temp_c = d[2];
  return true;
}

static const pin_t SCL = {GPIOB, 8};
static const pin_t SDA = {GPIOB, 9};

static void i2c_bus_clear(void) {
  pin_od(SCL, true); pin_od(SDA, true);
  pin_pull(SCL, 1); pin_pull(SDA, 1);
  pin_hi(SCL); pin_hi(SDA);
  pin_mode(SCL, 1); pin_mode(SDA, 1);
  for (int i = 0; i < 9; i++) { pin_lo(SCL); delay_us(5); pin_hi(SCL); delay_us(5); }
```

```
  pin_lo(SDA); delay_us(5); pin_hi(SCL); delay_us(5); pin_hi(SDA); delay_us(5);
}

static void i2c_init(void) {
  i2c_bus_clear();
  pin_af(SCL, 4); pin_af(SDA, 4);
  SCL.port->OSPEEDR |= (3U << (SCL.pin * 2)) | (3U << (SDA.pin * 2));
  pin_mode(SCL, 2); pin_mode(SDA, 2);
  RCC->APB1RSTR |= RCC_APB1RSTR_I2C1RST;
  RCC->APB1RSTR &= ~RCC_APB1RSTR_I2C1RST;
  I2C1->CR2 = 16;
  I2C1->CCR = 80;
  I2C1->TRISE = 17;
  I2C1->CR1 = I2C_CR1_PE;
}

static bool i2c_wait(uint32_t flag) {
  uint32_t s = micros();
  while (!(I2C1->SR1 & flag)) {
    if (I2C1->SR1 & I2C_SR1_AF) return false;
    if (micros() - s > I2C_TIMEOUT_US) return false;
  }
  return true;
}

static bool i2c_write(uint8_t addr7, uint8_t ctrl, const uint8_t *buf, uint32_t len) {
  uint32_t s = micros();
  while (I2C1->SR2 & I2C_SR2_BUSY)
    if (micros() - s > I2C_TIMEOUT_US) { i2c_init(); return false; }
  I2C1->CR1 |= I2C_CR1_START;
  if (!i2c_wait(I2C_SR1_SB)) goto fail;
  I2C1->DR = (uint8_t)(addr7 << 1);
  if (!i2c_wait(I2C_SR1_ADDR)) goto fail;
  (void)I2C1->SR1; (void)I2C1->SR2;
  if (!i2c_wait(I2C_SR1_TXE)) goto fail;
  I2C1->DR = ctrl;
  for (uint32_t i = 0; i < len; i++) {
    if (!i2c_wait(I2C_SR1_TXE)) goto fail;
    I2C1->DR = buf[i];
  }
  if (!i2c_wait(I2C_SR1_BTF)) goto fail;
  I2C1->CR1 |= I2C_CR1_STOP;
  return true;
fail:
  I2C1->SR1 &= ~I2C_SR1_AF;
  I2C1->CR1 |= I2C_CR1_STOP;
  return false;
}

static uint8_t oled_addr = 0;
static const uint8_t font5x7[][5] = {
#include "font_table.inc"
};

static bool oled_cmd(uint8_t c) { return i2c_write(oled_addr, 0x00, &c, 1); }

static bool oled_probe_and_init(void) {
  static const uint8_t addrs[2] = {0x3C, 0x3D};
  static const uint8_t init[] = {
    0xAE, 0xD5, 0x80, 0xA8, 0x3F, 0xD3, 0x00, 0x40, 0xAD, 0x8B,
    0xA1, 0xC8, 0xDA, 0x12, 0x81, 0xCF, 0xD9, 0x1F, 0xDB, 0x40,
    0xA4, 0xA6, 0xAF
  };
  oled_addr = 0;
  for (int a = 0; a < 2; a++) {
    uint8_t off = 0xAE;
    if (i2c_write(addrs[a], 0x00, &off, 1)) { oled_addr = addrs[a]; break; }
  }
  if (!oled_addr) return false;
  for (uint32_t i = 0; i < sizeof(init); i++)
    if (!oled_cmd(init[i])) { oled_addr = 0; return false; }
  return true;
}

static int glyph_index(char c) {
  if (c >= 'a' && c <= 'z' && c != 'c' && c != 'm') c -= 32;
  if (c >= ' ' && c <= 'Z') return c - ' ';
  if (c == 'c') return 59;
  if (c == 'm') return 60;
  return '?' - ' ';
}

```

```
313 static void oled_line(uint8_t page, const char *s) {
314   if (!oled_addr) return;
315   uint8_t buf[128] = {0};
316   for (int ch = 0; ch < 21; ch++) {
317     char c = *s ? *s++ : ' ';
318     const uint8_t *g = font5x7[glyph_index(c)];
319     for (int k = 0; k < 5; k++) buf[ch * 6 + k] = g[k];
320   }
321   oled_cmd(0xB0 | page);
322   oled_cmd(0x02);
323   oled_cmd(0x10);
324   if (!i2c_write(oled_addr, 0x40, buf, 128)) oled_addr = 0;
325 }
326
327 static char *put_str(char *b, const char *s) { while (*s) *b++ = *s++; return b; }
328 static char *put_u(char *b, uint32_t v, int width) {
329   char t[10]; int n = 0;
330   do { t[n++] = '0' + v % 10; v /= 10; } while (v);
331   for (int i = n; i < width; i++) *b++ = ' ';
332   while (n--) *b++ = t[n];
333   return b;
334 }
335
336 static char *put_u2(char *b, uint32_t v) { *b++ = (char)('0' + v / 10 % 10); *b++ = (char)('0'
     + v % 10); return b; }
337
338 static void zone_line(char *b, char zone, uint16_t cm, bool occ, int mode) {
339   *b++ = zone; *b++ = ':';
340   if (cm == DIST_NO_ECHO) b = put_str(b, "NO ECHO ");
341   else if (cm == DIST_FAR) b = put_str(b, "FAR     ");
342   else { b = put_u(b, cm, 3); b = put_str(b, "cm "); b = put_str(b, occ ? "OCC " : "--- "); }
343   b = put_str(b, MODE_TXT[mode]);
344   *b = 0;
345 }
346
347 /* TIM3_CH2 on PC7 (D9, zone A LED), TIM4_CH1 on PB6 (D10, zone B LED). 1kHz PWM. */
348 static void pwm_init_leds(void) {
349   RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN | RCC_AHB1ENR_GPIOCEN;
350   RCC->APB1ENR |= RCC_APB1ENR_TIM3EN | RCC_APB1ENR_TIM4EN;
351   pin_t a = {GPIOC, 7}, b = {GPIOB, 6};
352   pin_af(a, 2); pin_mode(a, 2);
353   pin_af(b, 2); pin_mode(b, 2);
354
355   TIM3->PSC = 16 - 1;
356   TIM3->ARR = 999;
357   TIM3->CCMR1 = (TIM3->CCMR1 & ~TIM_CCMR1_OC2M) | (6U << 12) | TIM_CCMR1_OC2PE;
358   TIM3->CCER |= TIM_CCER_CC2E;
359   TIM3->CR1 |= TIM_CR1_ARPE | TIM_CR1_CEN;
360
361   TIM4->PSC = 16 - 1;
362   TIM4->ARR = 999;
363   TIM4->CCMR1 = (TIM4->CCMR1 & ~TIM_CCMR1_OC1M) | (6U << 4) | TIM_CCMR1_OC1PE;
364   TIM4->CCER |= TIM_CCER_CC1E;
365   TIM4->CR1 |= TIM_CR1_ARPE | TIM_CR1_CEN;
366 }
367 static inline void led_set(int z, uint16_t duty) {
368   if (z == 0) TIM3->CCR2 = duty; else TIM4->CCR1 = duty;
369 }
370
371 static inline bool is_dark(void) { return pin_rd(LDR); }   /* HIGH = dark on this module */
372
373 /* ================================================================== */
374 /* Fan: TIM1_CH1 on PA8 (D7). 16 MHz / 800 = 20 kHz, above hearing, so  */
375 /* the motor doesn't whine. CCR 800 = 100 %.                            */
376 /* ================================================================== */
377 static void fan_init(void) {
378   RCC->APB2ENR |= RCC_APB2ENR_TIM1EN;
379   pin_t a = {GPIOA, 8};
380   pin_af(a, 1); pin_mode(a, 2);                    /* AF1 = TIM1 */
381   TIM1->PSC = 0;
382   TIM1->ARR = 799;
383   TIM1->CCMR1 = (TIM1->CCMR1 & ~TIM_CCMR1_OC1M) | (6U << 4) | TIM_CCMR1_OC1PE;
384   TIM1->CCR1 = 0;
385   TIM1->CCER |= TIM_CCER_CC1E;
386   TIM1->BDTR |= TIM_BDTR_MOE;                      /* advanced timer: no output without MOE
     */
387   TIM1->EGR = TIM_EGR_UG;
388   TIM1->CR1 |= TIM_CR1_ARPE | TIM_CR1_CEN;
389 }
390 static inline void fan_set(uint8_t pct) { TIM1->CCR1 = fan_ccr(pct, FAN_MAX_PCT); }
391
392 /* ================================================================== */
```

```
/* Pot: ADC1 channel 0 on PA0.                                          */
/* ==================================================================== */
#if USE_POT || USE_FAN_POT
static void adc_init(void) {
  RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;
  pin_t a0 = {GPIOA, 0}, a2 = {GPIOA, 4};
  pin_mode(a0, 3); pin_mode(a2, 3);                 /* analog */
  ADC1->SMPR2 = (6U << 0) | (6U << 12);             /* ch0, ch4: 144 cycles, fine for 10k pots */
  ADC1->SQR1 = 0;                                   /* 1 conversion per sequence */
  ADC1->CR2 = ADC_CR2_ADON;
  delay_us(20);
}
static bool adc_read(uint8_t ch, uint16_t *out) {
  ADC1->SQR3 = ch;                                  /* the one conversion is this channel */
  uint32_t sum = 0;
  for (int i = 0; i < POT_SAMPLES; i++) {
    ADC1->CR2 |= ADC_CR2_SWSTART;
    uint32_t s = micros();
    while (!(ADC1->SR & ADC_SR_EOC)) if (micros() - s > 200) return false;
    sum += ADC1->DR;                                /* reading DR clears EOC */
  }
  *out = (uint16_t)(sum / POT_SAMPLES);
  return true;
}
#endif

/* ==================================================================== */
/* RTC: on-chip, in the backup domain, keeps running through a reset.   */
/* ==================================================================== */
static bool rtc_lse;                                /* true = crystal, false = LSI fallback */
static bool rtc_ok;

static void rtc_unlock(void) { RTC->WPR = 0xCA; RTC->WPR = 0x53; }
static void rtc_lock(void)   { RTC->WPR = 0xFF; }

static bool rtc_wait_sync(void) {                   /* shadow registers caught up */
  rtc_unlock();
  RTC->ISR &= ~RTC_ISR_RSF;
  rtc_lock();
  uint32_t s = micros();
  while (!(RTC->ISR & RTC_ISR_RSF)) if (micros() - s > 50000) return false;
  return true;
}

static bool rtc_write(const dt_t *t) {
  if (!dt_valid(t)) return false;
  rtc_unlock();
  RTC->ISR |= RTC_ISR_INIT;
  uint32_t s = micros();
  while (!(RTC->ISR & RTC_ISR_INITF))
    if (micros() - s > 50000) { rtc_lock(); return false; }
  /* 32768 Hz / 128 / 256 = 1 Hz. LSI is ~32 kHz: / 128 / 250. Two writes, S then A. */
  RTC->PRER = rtc_lse ? 255U : 249U;
  RTC->PRER |= (127U << 16);
  RTC->TR = ((uint32_t)bcd(t->h) << 16) | ((uint32_t)bcd(t->m) << 8) | bcd(t->s);
  RTC->DR = ((uint32_t)bcd(t->y) << 16) | ((uint32_t)weekday(t->d, t->mo, 2000 + t->y) << 13) |
            ((uint32_t)bcd(t->mo) << 8) | bcd(t->d);
  RTC->CR &= ~RTC_CR_FMT;                           /* 24 hour */
  RTC->ISR &= ~RTC_ISR_INIT;
  rtc_lock();
  return rtc_wait_sync();
}

static void rtc_read(dt_t *t) {
  uint32_t tr = RTC->TR;                            /* reading TR freezes DR until DR is read */
  uint32_t dr = RTC->DR;
  t->h  = unbcd((uint8_t)((tr >> 16) & 0x3F));
  t->m  = unbcd((uint8_t)((tr >> 8) & 0x7F));
  t->s  = unbcd((uint8_t)(tr & 0x7F));
  t->y  = unbcd((uint8_t)((dr >> 16) & 0xFF));
  t->mo = unbcd((uint8_t)((dr >> 8) & 0x1F));
  t->d  = unbcd((uint8_t)(dr & 0x3F));
}

/* BKP0R holds a fingerprint of the build that last set the clock. Same build
 * (plain reset) = leave the running clock alone. New build or empty backup
 * domain (power was off) = start again from this build's compile time. */
static bool rtc_init(void) {
  const uint32_t magic = fnv1a(__DATE__, __TIME__);
```

```
  RCC->APB1ENR |= RCC_APB1ENR_PWREN;
  PWR->CR |= PWR_CR_DBP;                          /* allow writes to the backup domain */

  if ((RCC->BDCR & RCC_BDCR_RTCEN) && RTC->BKP0R == magic) {
    rtc_lse = (RCC->BDCR & RCC_BDCR_RTCSEL) == RCC_BDCR_RTCSEL_0;
    if (!rtc_lse) {                               /* LSI is switched off by every reset */
      RCC->CSR |= RCC_CSR_LSION;
      uint32_t s = micros();
      while (!(RCC->CSR & RCC_CSR_LSIRDY)) if (micros() - s > 10000) return false;
    }
    return rtc_wait_sync();
  }

  RCC->BDCR |= RCC_BDCR_BDRST;                    /* clean slate: only way to change RTCSEL */
  RCC->BDCR &= ~RCC_BDCR_BDRST;
  RCC->BDCR |= RCC_BDCR_LSEON;
  uint32_t s = micros();
  rtc_lse = true;
  while (!(RCC->BDCR & RCC_BDCR_LSERDY))
    if (micros() - s > 2500000) { rtc_lse = false; break; }   /* crystal can take ~2 s */

  if (rtc_lse) {
    RCC->BDCR |= RCC_BDCR_RTCSEL_0;               /* 01 = LSE */
  } else {
    RCC->BDCR &= ~RCC_BDCR_LSEON;
    RCC->CSR |= RCC_CSR_LSION;
    s = micros();
    while (!(RCC->CSR & RCC_CSR_LSIRDY)) if (micros() - s > 10000) return false;
    RCC->BDCR |= RCC_BDCR_RTCSEL_1;               /* 10 = LSI */
  }
  RCC->BDCR |= RCC_BDCR_RTCEN;

  dt_t t;
  if (!parse_build(__DATE__, __TIME__, &t) || !rtc_write(&t)) return false;
  RTC->BKP0R = magic;
  return true;
}

static void uart_put2(uint8_t v) { uart_putc((char)('0' + v / 10)); uart_putc((char)('0' + v % 10)); }
static void uart_time(void) {
  if (!rtc_ok) return;
  dt_t t; rtc_read(&t);
  uart_putc('['); uart_put2(t.h); uart_putc(':'); uart_put2(t.m); uart_putc(':'); uart_put2(t.s);
  uart_puts("] ");
}
static void uart_datetime(void) {
  if (!rtc_ok) { uart_puts("RTC not running\r\n"); return; }
  dt_t t; rtc_read(&t);
  uart_put2(t.d); uart_putc('/'); uart_put2(t.mo); uart_puts("/20"); uart_put2(t.y); uart_putc(' ');
  uart_put2(t.h); uart_putc(':'); uart_put2(t.m); uart_putc(':'); uart_put2(t.s);
  uart_puts(rtc_lse ? "  (crystal)\r\n" : "  (LSI fallback: drifts minutes per day)\r\n");
}

/* Serial commands, typed into PuTTY / Tera Term / Arduino serial monitor:
 *   T 08:30:00   set time       D 18/10/26   set date       ?   show both */
static void handle_command(char *cmd) {
  while (*cmd == ' ') cmd++;
  char op = *cmd;
  if (op >= 'a' && op <= 'z') op = (char)(op - 32);
  char *arg = cmd + 1;
  while (*arg == ' ') arg++;
  dt_t t;
  if (op == 'T' || op == 'D') {
    if (!rtc_ok) { uart_puts("RTC not running\r\n"); return; }
    rtc_read(&t);
    bool good = (op == 'T') ? parse_hms(arg, &t) : parse_dmy(arg, &t);
    if (good && rtc_write(&t)) { uart_puts("OK, now "); uart_datetime(); }
    else uart_puts(op == 'T' ? "Bad time. Use: T HH:MM or T HH:MM:SS\r\n"
                             : "Bad date. Use: D DD/MM/YY\r\n");
  } else if (op == '?' || op == 0) {
    uart_datetime();
  } else {
    uart_puts("Commands: T 08:30:00 | D 18/10/26 | ?\r\n");
  }
}

static void serial_service(void) {
  static char cmd[24];
```

```
  static uint8_t n;
  int c;
  while ((c = uart_getc()) >= 0) {
    if (c == '\r' || c == '\n') {
      if (n) { uart_puts("\r\n"); cmd[n] = 0; handle_command(cmd); n = 0; }
    } else if ((c == 8 || c == 127) && n) {
      n--; uart_puts("\b \b");
    } else if (c >= ' ' && n < sizeof(cmd) - 1) {
      cmd[n++] = (char)c; uart_putc((char)c);       /* echo, terminals don't by default */
    }
  }
}


/* ========================================================================== */
/* RFID: RC522 over SPI2, mode 0, 1 MHz. Register-level, no library.     */
/* ========================================================================== */
#define RC_COMMAND   0x01
#define RC_CONTROL   0x0C
#define RC_COMIRQ    0x04
#define RC_ERROR     0x06
#define RC_FIFODATA  0x09
#define RC_FIFOLEVEL 0x0A
#define RC_BITFRAME  0x0D
#define RC_COLL      0x0E
#define RC_MODE      0x11
#define RC_TXCONTROL 0x14
#define RC_TXASK     0x15
#define RC_TMODE     0x2A
#define RC_TPRESC    0x2B
#define RC_TRELOADH  0x2C
#define RC_TRELOADL  0x2D
#define RC_VERSION   0x37

#define RC_CMD_IDLE        0x00
#define RC_CMD_TRANSCEIVE  0x0C
#define RC_CMD_SOFTRESET   0x0F

#define PICC_REQA    0x26
#define PICC_SEL_CL1 0x93

typedef enum { RFID_NONE, RFID_OK, RFID_UNSUPPORTED } rfid_res_t;

static void spi2_init(void) {
  pin_t sck = {GPIOB, 13}, miso = {GPIOB, 14}, mosi = {GPIOB, 15};
  pin_af(sck, 5); pin_af(miso, 5); pin_af(mosi, 5);
  GPIOB->OSPEEDR |= (2U << (13 * 2)) | (2U << (14 * 2)) | (2U << (15 * 2));
  pin_mode(sck, 2); pin_mode(miso, 2); pin_mode(mosi, 2);
  pin_hi(RC_CS); pin_mode(RC_CS, 1);                /* idle high = not selected */
  pin_hi(RC_RST); pin_mode(RC_RST, 1);
  /* master, fPCLK/16 = 1 MHz, CPOL 0 CPHA 0, 8 bit, MSB first, software NSS */
  SPI2->CR1 = SPI_CR1_MSTR | (3U << 3) | SPI_CR1_SSM | SPI_CR1_SSI;
  SPI2->CR1 |= SPI_CR1_SPE;
}
static uint8_t spi2_xfer(uint8_t b) {
  uint32_t s = micros();
  while (!(SPI2->SR & SPI_SR_TXE))  if (micros() - s > 1000) return 0;
  SPI2->DR = b;
  while (!(SPI2->SR & SPI_SR_RXNE)) if (micros() - s > 1000) return 0;
  return (uint8_t)SPI2->DR;
}

/* Address byte: bit7 = 1 for read, bits 6..1 = register, bit0 = 0. */
static void rc_write(uint8_t reg, uint8_t v) {
  pin_lo(RC_CS);
  spi2_xfer((uint8_t)((reg << 1) & 0x7E));
  spi2_xfer(v);
  pin_hi(RC_CS);
}
static uint8_t rc_read(uint8_t reg) {
  pin_lo(RC_CS);
  spi2_xfer((uint8_t)(((reg << 1) & 0x7E) | 0x80));
  uint8_t v = spi2_xfer(0);
  pin_hi(RC_CS);
  return v;
}
static void rc_set(uint8_t reg, uint8_t mask) { rc_write(reg, rc_read(reg) | mask); }
static void rc_clr(uint8_t reg, uint8_t mask) { rc_write(reg, rc_read(reg) &
 (uint8_t)~mask); }

static uint8_t rc_version(void) { return rc_read(RC_VERSION); }
static bool rc_present(void) { uint8_t v = rc_version(); return v != 0x00 && v != 0xFF; }
```

```
631
632 static bool rc522_init(void) {
633   pin_lo(RC_RST); delay_ms(2);                        /* hard reset */
634   pin_hi(RC_RST); delay_ms(50);                       /* oscillator start-up */
635   rc_write(RC_COMMAND, RC_CMD_SOFTRESET);
636   delay_ms(10);
637   if (!rc_present()) return false;
638   rc_write(RC_TMODE, 0x80);                           /* timer starts itself after each
     transmit */
639   rc_write(RC_TPRESC, 0xA9);                          /* 13.56 MHz / (2*169+1) = 40 kHz, 25 us
     ticks */
640   rc_write(RC_TRELOADH, 0x01);
641   rc_write(RC_TRELOADL, 0x40);                        /* 320 ticks = 8 ms: no answer by then =
     no card */
642   rc_write(RC_TXASK, 0x40);                           /* 100 % ASK modulation, required by ISO
     14443A */
643   rc_write(RC_MODE, 0x3D);                            /* CRC preset 0x6363 */
644   rc_set(RC_TXCONTROL, 0x03);                         /* antenna on */
645   return true;
646 }
647
648 /* Send tx[] to the card, wait for the answer. txbits = number of valid bits in
649  * the last tx byte (0 = all 8, REQA uses 7). Returns false on timeout/error. */
650 static bool rc_transceive(const uint8_t *tx, uint8_t txlen, uint8_t txbits,
651                           uint8_t *rx, uint8_t *rxlen, uint8_t *rxbits) {
652   uint8_t rxmax = *rxlen;
653   rc_write(RC_COMMAND, RC_CMD_IDLE);
654   rc_write(RC_COMIRQ, 0x7F);                          /* clear every pending flag */
655   rc_write(RC_FIFOLEVEL, 0x80);                       /* flush FIFO */
656   for (uint8_t i = 0; i < txlen; i++) rc_write(RC_FIFODATA, tx[i]);
657   rc_write(RC_BITFRAME, txbits);
658   rc_write(RC_COMMAND, RC_CMD_TRANSCEIVE);
659   rc_set(RC_BITFRAME, 0x80);                          /* StartSend */
660
661   uint32_t s = micros();
662   for (;;) {
663     uint8_t irq = rc_read(RC_COMIRQ);
664     if (irq & 0x30) break;                            /* RxIRq or IdleIRq: got something */
665     if (irq & 0x01) return false;                     /* TimerIRq: nobody answered */
666     if (micros() - s > 15000) return false;           /* belt and braces */
667   }
668   if (rc_read(RC_ERROR) & 0x1B) return false;         /* buffer overflow, collision, parity,
     protocol */
669   uint8_t n = rc_read(RC_FIFOLEVEL);
670   if (n > rxmax) n = rxmax;
671   for (uint8_t i = 0; i < n; i++) rx[i] = rc_read(RC_FIFODATA);
672   *rxlen = n;
673   *rxbits = rc_read(RC_CONTROL) & 0x07;                    /* ControlReg: valid bits in last
     byte */
674   return true;
675 }
676
677 /* A card that is IDLE answers REQA with its 2-byte ATQA. Cards that have
678  * already been read ignore REQA until they leave the field, which is exactly
679  * the "one tap, one event" behaviour we want. */
680 static bool rc_reqa(void) {
681   uint8_t cmd = PICC_REQA, rx[2], n = 2, bits = 0;
682   rc_clr(RC_COLL, 0x80);
683   return rc_transceive(&cmd, 1, 7, rx, &n, &bits) && n == 2 && bits == 0;
684 }
685
686 /* Anticollision, cascade level 1: ask for the 4 UID bytes + BCC check byte. */
687 static rfid_res_t rc_read_uid(uint32_t *uid) {
688   uint8_t cmd[2] = { PICC_SEL_CL1, 0x20 }, rx[5], n = 5, bits = 0;
689   if (!rc_transceive(cmd, 2, 0, rx, &n, &bits) || n != 5 || bits != 0) return RFID_NONE;
690   if ((uint8_t)(rx[0] ^ rx[1] ^ rx[2] ^ rx[3]) != rx[4]) return RFID_NONE;
691   if (rx[0] == 0x88) return RFID_UNSUPPORTED;         /* cascade tag: 7 or 10 byte UID, not
     handled */
692   *uid = ((uint32_t)rx[0] << 24) | ((uint32_t)rx[1] << 16) | ((uint32_t)rx[2] << 8) | rx[3];
693   if (*uid == 0 || *uid == 0xFFFFFFFFUL) return RFID_NONE;   /* bus noise, not a card */
694   return RFID_OK;
695 }
696
697 /* ---- door state, updated by rfid_service() from anywhere in the loop ---- */
698 static struct {
699   roster_t roster;
700   bool     ok;               /* reader answered the last probe */
701   uint32_t last_uid;         /* 0 = nothing yet */
702   const char *last_txt;      /* "IN", "OUT", "DENY", "FULL", "7B" */
703   bool     boost_on;
704   uint32_t boost_t0;
705 } rf = { .last_txt = "" };
```

```
706
707 static inline int  rf_count(void) { return roster_count(&rf.roster); }
708 static bool rf_boost(void) {
709   if (rf.boost_on && micros() - rf.boost_t0 >= ENTRY_BOOST_MS * 1000UL) rf.boost_on = false;
710   return rf.boost_on;
711 }
712
713 static void uart_puthex32(uint32_t v) {
714   for (int i = 3; i >= 0; i--) uart_puthex8((uint8_t)(v >> (i * 8)));
715 }
716
717 /* Poll once. Cheap when nobody is there (about 8 ms for the timeout). */
718 static void rfid_service(void) {
719   if (!rf.ok) return;
720   if (!rc_reqa()) return;
721   uint32_t uid = 0;
722   rfid_res_t r = rc_read_uid(&uid);
723   if (r == RFID_NONE) return;
724   if (r == RFID_UNSUPPORTED) {
725     rf.last_uid = 0; rf.last_txt = "7B";
726     uart_puts("RFID: 7/10-byte UID card, not supported\r\n");
727     return;
728   }
729   rf.last_uid = uid;
730   uart_time(); uart_puts("RFID "); uart_puthex32(uid); uart_puts(" ");
731   if (!uid_allowed(ALLOWED_UIDS, uid)) {
732     rf.last_txt = "DENY";
733     uart_puts("DENIED\r\n");
734     return;
735   }
736   switch (roster_tap(&rf.roster, uid, micros(), TAP_LOCKOUT_MS * 1000UL)) {
737     case TAP_IN:
738       rf.last_txt = "IN";
739       rf.boost_on = true; rf.boost_t0 = micros();
740       uart_puts("IN");
741       break;
742     case TAP_OUT:
743       rf.last_txt = "OUT";
744       if (rf_count() == 0) rf.boost_on = false;
745       uart_puts("OUT");
746       break;
747     case TAP_FULL:
748       rf.last_txt = "FULL";
749       uart_puts("ROSTER FULL");
750       break;
751     default:
752       return;                                          /* same card still on the reader */
753   }
754   uart_puts("  inside="); uart_putu((uint32_t)rf_count()); uart_puts("\r\n");
755 }
756
757 /* delay_ms that keeps listening for cards while it waits */
758 static void wait_poll(uint32_t ms) {
759   uint32_t s = micros(), us = ms * 1000UL;
760   while (micros() - s < us) { rfid_service(); serial_service(); delay_ms(2); }
761 }
762
763 static void setup(void) {
764   RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOBEN | RCC_AHB1ENR_GPIOCEN;
765   RCC->APB1ENR |= RCC_APB1ENR_TIM2EN | RCC_APB1ENR_I2C1EN | RCC_APB1ENR_USART2EN |
766                   RCC_APB1ENR_SPI2EN;
767
768   TIM2->PSC = 16 - 1;
769   TIM2->ARR = 0xFFFFFFFF;
770   TIM2->EGR = TIM_EGR_UG;
771   TIM2->CR1 = TIM_CR1_CEN;
772
773   for (int z = 0; z < 2; z++) {
774     pin_lo(TRIG[z]); pin_mode(TRIG[z], 1);
775     pin_mode(ECHO[z], 0); pin_pull(ECHO[z], 2);
776   }
777   pin_od(DHT, true); pin_pull(DHT, 1); pin_hi(DHT); pin_mode(DHT, 1);
778   pin_mode(LED, 1);
779   pin_mode(LDR, 0); pin_pull(LDR, 1);   /* pull-up, same reasoning as the arduino build */
780
781   pwm_init_leds();
782   led_set(0, 0); led_set(1, 0);
783   fan_init();                                          /* motors off from the first instant */
784
785   uart_init();
786   uart_puts("\r\nSmart Classroom - Nucleo-F401RE\r\n");
787
```

```
788   rtc_ok = rtc_init();
789   uart_puts("RTC: "); uart_datetime();
790   if (rtc_ok && !rtc_lse) uart_puts("RTC: no 32.768 kHz crystal answered, using LSI\r\n");
791 #if USE_POT || USE_FAN_POT
792   adc_init();
793 #endif
794   uart_rx_init();
795   button_init();
796   uart_puts("Commands: T 08:30:00 | D 18/10/26 | ?\r\n");
797
798   i2c_init();
799   delay_ms(100);
800   if (oled_probe_and_init()) {
801     uart_puts("OLED found at 0x"); uart_puthex8(oled_addr); uart_puts("\r\n");
802     for (uint8_t p = 0; p < 8; p++) oled_line(p, "");
803   } else {
804     uart_puts("OLED: no ACK - check wiring\r\n");
805   }
806
807   spi2_init();
808   rf.ok = rc522_init();
809   if (rf.ok) { uart_puts("RC522 found, version 0x"); uart_puthex8(rc_version()); uart_puts("\
    r\n"); }
810   else uart_puts("RC522: no answer - check wiring (3V3 only!)\r\n");
811   delay_ms(1000);
812 }
813
814 static char *put_hex8(char *b, uint8_t v) {
815   const char *h = "0123456789ABCDEF";
816   *b++ = h[v >> 4]; *b++ = h[v & 15];
817   return b;
818 }
819 static char *put_hex32(char *b, uint32_t v) {
820   const char *h = "0123456789ABCDEF";
821   for (int i = 7; i >= 0; i--) *b++ = h[(v >> (i * 4)) & 15];
822   return b;
823 }
824
825 int main(void) {
826   setup();
827
828   uint8_t temp = 0, hum = 0;
829   bool dht_ok = false;
830   fan_t fan = {0, false, 0};
831   uint8_t fan_now = 0;
832   uint8_t knob = 100;                              /* fan speed from the A2 pot; 100 if no
    pot */
833   uint32_t last_dht = micros() - DHT_PERIOD_US;
834   uint16_t cm[2];
835   bool occ[2] = {false, false};
836   uint32_t pending_since[2] = {0, 0};
837   bool stale_timing = false;
838   uint32_t stale_since = 0;
839   char line[24];
840   ovr_t ovr = OVR_AUTO;
841   uint32_t btn_seen = btn_presses;
842   uint16_t bright = BRIGHT_MAX;
843   static const char *OVR_TXT[3] = { "AUTO", "ON", "OFF" };
844
845   for (;;) {
846     for (int z = 0; z < 2; z++) { cm[z] = read_cm(z); wait_poll(PING_GAP_MS); }
847
848     if (micros() - last_dht >= DHT_PERIOD_US) {
849       last_dht = micros();
850       dht_ok = dht11_read(&temp, &hum);
851       if (!oled_addr && oled_probe_and_init())
852         for (uint8_t p = 0; p < 8; p++) oled_line(p, "");
853       rf.ok = rf.ok ? rc_present() : rc522_init();   /* notice a reader that was unplugged or
    plugged in */
854     }
855
856     serial_service();
857
858     uint32_t presses = btn_presses;                /* one read: the ISR may bump it any time
    */
859     if (presses != btn_seen) {
860       ovr = (ovr_t)((ovr + (presses - btn_seen)) % 3);
861       btn_seen = presses;
862       uart_time(); uart_puts("Button: override "); uart_puts(OVR_TXT[ovr]); uart_puts("\r\n");
863     }
864 #if USE_POT
865     { uint16_t v; if (adc_read(0, &v)) bright = pot_brightness(v, bright); }
```

```
#endif
#if USE_FAN_POT
    { uint16_t v; if (adc_read(4, &v)) knob = fan_knob(v, knob); }
#endif

    bool dark = is_dark();
    bool boost = rf_boost();
    int occ_n = 0;
    for (int z = 0; z < 2; z++) {
      bool raw = (cm[z] != DIST_NO_ECHO && cm[z] != DIST_FAR && cm[z] < OCC_CM);
      if (raw != occ[z]) {
        if (pending_since[z] == 0) pending_since[z] = micros();
        uint32_t need_us = (raw ? OCC_MS : VACANT_MS) * 1000UL;
        if (micros() - pending_since[z] >= need_us) {
          occ[z] = raw;
          pending_since[z] = 0;
        }
      } else {
        pending_since[z] = 0;
      }
      if (occ[z]) occ_n++;
      bool occ_eff = occ[z] || boost;               /* a recent tap-in counts as "someone is here" */


      led_set(z, led_duty(ovr, mode[z], occ_eff, dark, bright, LED_DIM));
    }

    /* Fan: on while at least one card is tapped in, speed from the knob. */
    fan_now = fan_cmd(ovr, rf_count() > 0, knob);
    fan_set(fan_apply(&fan, fan_now, micros()));

    /* Everyone tapped in but the room has been empty for ages: they left without tapping out. */
    if (rf_count() > 0 && occ_n == 0 && !boost) {
      if (!stale_timing) { stale_timing = true; stale_since = micros(); }
      else if (micros() - stale_since >= STALE_RESET_MS * 1000UL) {
        roster_clear(&rf.roster);
        rf.last_txt = "RESET";
        stale_timing = false;
        uart_puts("RFID roster cleared: zones empty too long\r\n");
      }
    } else {
      stale_timing = false;
    }

    oled_line(0, dark ? "LIGHT: DARK" : "LIGHT: BRIGHT");
    rfid_service();
    {
      char *b = put_str(line, "IN:"); b = put_u(b, (uint32_t)rf_count(), 1); b = put_str(b, " ");
      b = rf.last_uid ? put_hex32(b, rf.last_uid) : put_str(b, "--------");
      b = put_str(b, " "); b = put_str(b, rf.last_txt); *b = 0;
    }
    oled_line(1, line);
    zone_line(line, 'A', cm[0], occ[0], mode[0]); oled_line(2, line);
    zone_line(line, 'B', cm[1], occ[1], mode[1]); oled_line(3, line);
    rfid_service();
    if (dht_ok) {
      char *b = line;
      b = put_str(b, "T:"); b = put_u(b, temp, 2); b = put_str(b, "C   H:");
      b = put_u(b, hum, 2); b = put_str(b, "%"); *b = 0;
    } else {
      put_str(line, "DHT11 ERROR")[0] = 0;
    }
    oled_line(5, line);
    rfid_service();
    if (rtc_ok) {
      dt_t t; rtc_read(&t);
      char *b = put_u2(line, t.d); *b++ = '/'; b = put_u2(b, t.mo); *b++ = ' ';
      b = put_u2(b, t.h); *b++ = ':'; b = put_u2(b, t.m); *b++ = ':'; b = put_u2(b, t.s);
      b = put_str(b, rtc_lse ? "" : " LSI"); *b = 0;
    } else {
      put_str(line, "RTC ERROR")[0] = 0;
    }
    oled_line(4, line);
    {
      /* "OCC1/2 AUTO L75 F60": occupancy, override, light %, fan % (0 = off) */
      char *b = put_str(line, "OCC"); b = put_u(b, occ_n, 1); b = put_str(b, "/2 ");
      b = put_str(b, OVR_TXT[ovr]); b = put_str(b, " L");
      b = put_u(b, (uint32_t)bright * 100 / BRIGHT_MAX, 1); b = put_str(b, " F");
      b = put_u(b, fan_now, 1); *b = 0;
```

```
945      }
946      oled_line(6, line);
947      if (rf.ok) { char *b = put_str(line, "RFID OK V"); b = put_hex8(b, rc_version()); *b =
     0; }
948      else put_str(line, "RFID NOT FOUND")[0] = 0;
949      oled_line(7, line);
950      rfid_service();
951
952      serial_service();
953      if (rx_any && micros() - rx_last_us < RX_QUIET_US) goto heartbeat;   /* you are typing:
      keep the terminal clean */
954
955      uart_time();
956      const char *zn = "AB";
957      for (int z = 0; z < 2; z++) {
958        uart_putc(zn[z]); uart_putc('=');
959        if (cm[z] == DIST_NO_ECHO) uart_puts("NOECHO");
960        else if (cm[z] == DIST_FAR) uart_puts("FAR");
961        else uart_putu(cm[z]);
962        uart_puts(occ[z] ? "(OCC) " : "      ");
963        uart_puts(MODE_TXT[mode[z]]); uart_puts("  ");
964      }
965      uart_puts("LDR="); uart_puts(dark ? "DARK" : "BRIGHT"); uart_puts("  ");
966      if (dht_ok) { uart_puts("T="); uart_putu(temp); uart_puts(" H="); uart_putu(hum); }
967      else uart_puts("DHT=ERR");
968      uart_puts("  IN="); uart_putu((uint32_t)rf_count()); uart_puts(boost ? "(boost)" : "");
969      uart_puts(rf.ok ? "  RFID=ok" : "  RFID=none");
970      uart_puts("  OVR="); uart_puts(OVR_TXT[ovr]);
971      uart_puts(" BRT="); uart_putu((uint32_t)bright * 100 / BRIGHT_MAX); uart_putc('%');
972      uart_puts(" FAN="); uart_putu(fan_now); uart_putc('%');
973      uart_puts(oled_addr ? "  OLED=ok\r\n" : "  OLED=none\r\n");
974
975  heartbeat:
976      LED.port->ODR ^= (1U << LED.pin);
977    }
978  }
```

## Annexure C  Decision Logic: logic.h

Decision logic without hardware access, compiled into both the firmware and the host-based test program.

### logic.h

```
/*
 * logic.h - headcount, pot, and date/time logic, no hardware in it.
 *
 * Everything here is plain C on integers, so the same file is compiled into
 * the firmware AND into tests/host_test.c on a laptop. Time is always passed
 * in (as microseconds from TIM2) so tests can fake it.
 */
#ifndef LOGIC_H
#define LOGIC_H

#include <stdint.h>
#include <stdbool.h>

/* ------------------------------------------------------------- roster --
 * Who is inside the room, by card. A card seen for the first time is
 * "inside". Every later tap flips it. A card seen again within lockout_us
 * of its last sighting is ignored (and the window is refreshed), so a card
 * resting on the reader cannot toggle itself in and out.
 */
#define ROSTER_MAX 8

typedef struct { uint32_t uid; uint32_t last_us; bool used; bool inside; } rost_entry_t;
typedef struct { rost_entry_t e[ROSTER_MAX]; } roster_t;
typedef enum { TAP_IN, TAP_OUT, TAP_IGNORED, TAP_FULL } tap_t;

static inline void roster_clear(roster_t *r) {
  for (int i = 0; i < ROSTER_MAX; i++) r->e[i].used = false;
}

static inline int roster_count(const roster_t *r) {
  int n = 0;
  for (int i = 0; i < ROSTER_MAX; i++) if (r->e[i].used && r->e[i].inside) n++;
  return n;
}

static inline tap_t roster_tap(roster_t *r, uint32_t uid, uint32_t now_us, uint32_t
 lockout_us) {
  for (int i = 0; i < ROSTER_MAX; i++) {
    if (r->e[i].used && r->e[i].uid == uid) {
      bool locked = (now_us - r->e[i].last_us) < lockout_us;
      r->e[i].last_us = now_us;
      if (locked) return TAP_IGNORED;
      r->e[i].inside = !r->e[i].inside;
      return r->e[i].inside ? TAP_IN : TAP_OUT;
    }
  }
  /* new card: free slot first, else evict the card that left longest ago */
  int slot = -1;
  for (int i = 0; i < ROSTER_MAX; i++) if (!r->e[i].used) { slot = i; break; }
  if (slot < 0) {
    uint32_t oldest = 0;
    for (int i = 0; i < ROSTER_MAX; i++) {
      if (r->e[i].inside) continue;
      uint32_t age = now_us - r->e[i].last_us;
      if (slot < 0 || age > oldest) { slot = i; oldest = age; }
    }
  }
  if (slot < 0) return TAP_FULL;
  r->e[slot].used = true;
  r->e[slot].uid = uid;
  r->e[slot].inside = true;
  r->e[slot].last_us = now_us;
  return TAP_IN;
}

/* Zero-terminated allow list. An empty list ({0}) accepts every card. */
static inline bool uid_allowed(const uint32_t *list, uint32_t uid) {
  if (list[0] == 0) return true;
```

```
  for (; *list; list++) if (*list == uid) return true;
  return false;
}

/* ------------------------------------------------------------------ pot --
 * One pot dims both zones. 12-bit ADC -> ON brightness in PWM counts,
 * 50..999 (5..100 %). Never 0: switching off is the button's / mode's job.
 * Changes smaller than POT_STEP counts are ignored so ADC noise does not
 * make the LEDs shimmer.
 */
#define BRIGHT_MIN 50
#define BRIGHT_MAX 999
#define POT_STEP   8

static inline uint16_t pot_brightness(uint16_t adc, uint16_t cur) {
  if (adc > 4095) adc = 4095;
  int target = BRIGHT_MIN + (int)adc * (BRIGHT_MAX - BRIGHT_MIN) / 4095;
  int d = target - (int)cur;
  if (d < 0) d = -d;
  if (d < POT_STEP && target != BRIGHT_MIN && target != BRIGHT_MAX) return cur;
  return (uint16_t)target;
}

/* ---------------------------------------------------------- override --
 * The blue button cycles every LED through AUTO -> ON -> OFF -> AUTO.
 */
typedef enum { OVR_AUTO = 0, OVR_ON = 1, OVR_OFF = 2 } ovr_t;

static inline uint16_t led_duty(ovr_t ovr, int mode, bool occupied, bool dark,
                                uint16_t bright, uint16_t dim) {
  if (ovr == OVR_ON)  return bright;
  if (ovr == OVR_OFF) return 0;
  switch (mode) {
    case 0:  return 0;
    case 1:  return (occupied && dark) ? bright : 0;
    case 2:  return dim < bright ? dim : bright;
    default: return bright;
  }
}

/* ------------------------------------------------------------------ fan --
 * One fan for the room. The knob (fan pot) sets the speed; RFID decides
 * whether anyone is inside. Bottom of the knob's travel = off, after that
 * FAN_MIN_PCT..100 % (a hobby motor stalls below about a third).
 * Knob changes under FAN_KNOB_STEP are ignored so ADC noise doesn't hunt.
 */
#define FAN_MIN_PCT    35
#define FAN_KNOB_OFF   200      /* ADC counts: below this the knob means "off" */
#define FAN_KNOB_STEP  3

static inline uint8_t fan_knob(uint16_t adc, uint8_t cur) {
  if (adc > 4095) adc = 4095;
  if (adc < FAN_KNOB_OFF) return 0;
  int target = FAN_MIN_PCT + (int)(adc - FAN_KNOB_OFF) * (100 - FAN_MIN_PCT) / (4095 -
 FAN_KNOB_OFF);
  int d = target - (int)cur;
  if (d < 0) d = -d;
  if (cur != 0 && d < FAN_KNOB_STEP && target != FAN_MIN_PCT && target != 100) return cur;
  return (uint8_t)target;
}

/* Button override first (ON runs it even with nobody inside, OFF stops it),
 * otherwise: someone tapped in -> knob speed, nobody inside -> off. */
static inline uint8_t fan_cmd(ovr_t ovr, bool someone_inside, uint8_t knob) {
  if (ovr == OVR_OFF) return 0;
  if (ovr == OVR_ON)  return knob;
  return someone_inside ? knob : 0;
}

/* A stopped motor will not start on 35 %. When the command goes from 0 to
 * something, give it full allowed speed for FAN_KICK_US first. */
#define FAN_KICK_US 300000UL

typedef struct { uint8_t prev; bool kicking; uint32_t t0; } fan_t;

static inline uint8_t fan_apply(fan_t *f, uint8_t pct, uint32_t now_us) {
  if (pct == 0) { f->prev = 0; f->kicking = false; return 0; }
  if (f->prev == 0) { f->kicking = true; f->t0 = now_us; }
  f->prev = pct;
  if (f->kicking) {
    if (now_us - f->t0 < FAN_KICK_US) return 100;
    f->kicking = false;
```

```
  }
  return pct;
}

/* Percent -> timer compare value. max_pct caps the average motor voltage:
 * 9 V battery x 60 % ~ 5.4 V for 3-6 V hobby motors. ARR 799: 800 = always on. */
static inline uint16_t fan_ccr(uint8_t pct, uint8_t max_pct) {
  if (pct > 100) pct = 100;
  return (uint16_t)((uint32_t)pct * max_pct * 8 / 100);
}

/* ------------------------------------------------------------ date/time --
 */
typedef struct { uint8_t h, m, s, d, mo, y; } dt_t;   /* y = years since 2000 */

static inline uint8_t bcd(uint8_t v)   { return (uint8_t)(((v / 10) << 4) | (v % 10)); }
static inline uint8_t unbcd(uint8_t v) { return (uint8_t)((v >> 4) * 10 + (v & 15)); }

static inline int days_in_month(int mo, int year) {
  static const uint8_t dm[12] = {31,28,31,30,31,30,31,31,30,31,30,31};
  if (mo == 2 && ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)) return 29;
  return dm[mo - 1];
}

/* 1 = Monday .. 7 = Sunday, the RTC's convention (Sakamoto's method). */
static inline uint8_t weekday(int d, int mo, int year) {
  static const uint8_t t[12] = {0,3,2,5,0,3,5,1,4,6,2,4};
  if (mo < 3) year -= 1;
  int w = (year + year / 4 - year / 100 + year / 400 + t[mo - 1] + d) % 7;   /* 0 = Sunday */
  return (uint8_t)(w == 0 ? 7 : w);
}

static inline bool dt_valid(const dt_t *t) {
  return t->h < 24 && t->m < 60 && t->s < 60 && t->y < 100 &&
         t->mo >= 1 && t->mo <= 12 && t->d >= 1 && t->d <= days_in_month(t->mo, 2000 + t->y);
}

/* reads 1-2 digits, returns pointer past them or 0 */
static inline const char *read_num(const char *p, int *out, int maxdig) {
  int v = 0, n = 0;
  while (*p >= '0' && *p <= '9' && n < maxdig) { v = v * 10 + (*p - '0'); p++; n++; }
  if (!n) return 0;
  *out = v; return p;
}

/* "HH:MM" or "HH:MM:SS" */
static inline bool parse_hms(const char *p, dt_t *t) {
  int h, m, s = 0;
  if (!(p = read_num(p, &h, 2)) || *p++ != ':') return false;
  if (!(p = read_num(p, &m, 2))) return false;
  if (*p == ':') { if (!(p = read_num(p + 1, &s, 2))) return false; }
  if (*p != 0 || h > 23 || m > 59 || s > 59) return false;
  t->h = (uint8_t)h; t->m = (uint8_t)m; t->s = (uint8_t)s;
  return true;
}

/* "DD/MM/YY" or "DD/MM/YYYY" ('-' also accepted) */
static inline bool parse_dmy(const char *p, dt_t *t) {
  int d, mo, y;
  if (!(p = read_num(p, &d, 2)) || (*p != '/' && *p != '-')) return false;
  if (!(p = read_num(p + 1, &mo, 2)) || (*p != '/' && *p != '-')) return false;
  if (!(p = read_num(p + 1, &y, 4)) || *p != 0) return false;
  if (y >= 2000) y -= 2000;
  if (y < 0 || y > 99 || mo < 1 || mo > 12 || d < 1 || d > days_in_month(mo, 2000 + y)) return
 false;
  t->d = (uint8_t)d; t->mo = (uint8_t)mo; t->y = (uint8_t)y;
  return true;
}

/* The compiler's __DATE__ ("Oct 18 2026") and __TIME__ ("08:31:05"). */
static inline bool parse_build(const char *date, const char *time, dt_t *t) {
  static const char mon[] = "JanFebMarAprMayJunJulAugSepOctNovDec";
  int mo = 0;
  for (int i = 0; i < 12; i++)
    if (date[0] == mon[i*3] && date[1] == mon[i*3+1] && date[2] == mon[i*3+2]) mo = i + 1;
  if (!mo) return false;
  int d = (date[4] == ' ' ? 0 : date[4] - '0') * 10 + (date[5] - '0');
  int y = (date[9] - '0') * 10 + (date[10] - '0');
  t->d = (uint8_t)d; t->mo = (uint8_t)mo; t->y = (uint8_t)y;
  t->h = (uint8_t)((time[0] - '0') * 10 + time[1] - '0');
  t->m = (uint8_t)((time[3] - '0') * 10 + time[4] - '0');
  t->s = (uint8_t)((time[6] - '0') * 10 + time[7] - '0');
```

```
  return dt_valid(t);
}

/* FNV-1a, used to tell one build from the next */
static inline uint32_t fnv1a(const char *a, const char *b) {
  uint32_t h = 2166136261UL;
  for (; *a; a++) h = (h ^ (uint8_t)*a) * 16777619UL;
  for (; *b; b++) h = (h ^ (uint8_t)*b) * 16777619UL;
  return h | 1;                                  /* never 0: 0 means "backup domain empty" */
}

#endif
```

**Plagiarism Report**

**Note: Should be checked for "*exclude matches* < 3 words"**

First page of the plagiarized report which include

**Submission date:**

**Submission ID:**

**File name:**

**Word count:**

**Character count:**

Next page onwards:

**Originality report page followed by remaining pages after that.**